INFEARA MEDICAL SYSTEM (IMS)

A PROJECT REPORT SUBMITTED TO

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

BACHELOR OF COMPUTER APPLICATIONS

BY

VIGNESH M

REG. NO. DC2431006010004

UNDER THE GUIDANCE OF

Dr. A. THIRUMURTHI RAJA M.C.A., M.E., PH.D.,

ASSISTANT PROFESSOR,



DEPARTMENT OF COMPUTER APPLICATIONS

DIRECTORATE OF DISTANCE EDUCATION

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be university u/s 3 of UGC Act, 1956)

Kattankulathur – 603 203

Chennai, Tamilnadu

NOV 2026

BONAFIDE CERTIFICATE

This is to certify that the project report titled **“INFEARA MEDICAL SYSTEM (IMS)”** is a bonafide work carried out by **VIGNESH M** (**DC2431006010004**) under my supervision for the award of the Degree of Bachelor of Computer Applications. To my knowledge the work reported herein is the original work done by the candidate.

PROJECT GUIDE

Dr.A.Thirumurthi Raja

Assistant Professor,

SRMIST-DDE,

HEAD OF DEPARTMENT

Dr.A.Thirumurthi Raja

Program Coordinator

Department of Computer Applications,

Directorate of Distance Education (DDE)

SRMIST,

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

With profound gratitude to the ALMIGHTY, I take this chance to thank the people who helped me to complete this project.

I take this as a right opportunity to say THANKS to my parents who are there to stand with me always with the words “ANYTHING IS ACHIEVABLE WITH HARDWORK AND PERSISTENCE”.

I am thankful to **Dr.T.R. Paarivendhar, Chancellor**, SRM Institute of Science & Technology who gave us the platform to establish ourselves and reach greater heights.

I express my deep sense of gratitude to the Director, Directorate of Distance Education, SRM Institute of Science & Technology for their wholehearted support and encouragement.

With gratitude I like to thank my project guide **Dr.A.Thirumurthi Raja**, Assistant Professor, Department of Computer Applications (DDE), for his help, support, encouragement, suggestions, and guidance throughout the development phases of the project.

I take this opportunity to extend my sincere gratitude to all the faculty members of the department who provided their support through valuable comments and suggestions during the reviews.

A great note of gratitude to friends and colleagues who helped in carrying out this project work to a successful completion.

Vignesh M

TABLE OF CONTENTS

1. CHAPTER 1 –

1.0 INTRODUCTION 8

1.1 EXISTING SYSTEM 9

1.2 PROPOSED SYSTEM 10

1.3 SYSTEM SPECIFICATION 13

1.3.1 HARDWARE REQUIREMENTS

1.3.2 SOFTWARE REQUIREMENTS

1.4 ARCHITECTURE DIAGRAM 14

1.5 UML DIAGRAMS 16

1.5.1 USE CASE DIAGRAM 16

1.5.2 CLASS DIAGRAM 17

1.5.3 SEQUENCE DIAGRAM 18

1.5.4 ACTIVITY DIAGRAM 19

1.5.5 DATAFLOW DIAGRAM 20

1.5.6 ENTITY RELATIONSHIP DIAGRAM 23

2. CHAPTER 2 –

2.1 LITERATURE SURVEY 25

3. CHAPTER 3 –

3.0 MODULES 31

3.1 MODULE DESCRIPTION 31

3.1.1 SUPER ADMIN & TENANT ENROLLMENT 31

3.1.2 IDENTITY, AUTH TOKENS & SECURITY 32

3.1.3 CLINIC ADMIN & PATIENT LINKING 33

3.1.4 DOCTOR SCHEDULE & SLOT BOOKING 34

3.1.5 CLINICAL CONSULTATION & ADDENDA 34

3.1.6 ELECTRONIC PRESCRIPTION (E-RX) 35
3.1.7 PHARMACY POS & FINANCE LEDGER 36
3.1.8 TRANSACTIONAL OUTBOX & AUDIT LOGGING 37
4. CHAPTER 4 –
4.0 LANGUAGE SPECIFICATIONS 38
5. CHAPTER 5 –
5.1 SYSTEM TESTING 51
5.1.1 SYSTEM TESTING AND MAINTENANCE 52
5.2 UNIT TESTING 53
5.3 INTEGRATION TESTING 54
6. CHAPTER 6 –
6.1 CONCLUSION 58
6.2 SCREENSHOTS 60
6.3 CODING 67
6.4 REFERENCES 103

Infeara Medical System (IMS): Multi-Tenant Clinical, e-Prescription and Pharmacy POS Platform

Abstract:

In contemporary outpatient clinics, diagnostic centers, and retail community pharmacies, clinical record management, patient identity verification, prescription authoring, and medication dispensing routinely operate across disconnected software silos and handwritten registers. This operational fragmentation produces duplicate walk-in versus online patient charts, double-booked physician appointment slots, forged or illegible paper prescriptions, accidental over-dispensing across pharmacy counters, and manual errors in daily financial reconciliation. This project addresses these critical healthcare challenges by designing and implementing the Infeara Medical System (IMS)—a unified, multi-tenant clinical operations, electronic prescribing (e-Rx), and pharmacy Point of Sale (POS) platform built on Java 27, Spring Boot 4.1.2, Spring Security 6, Thymeleaf 3.1, Flyway, and Microsoft SQL Server 2022.

The platform orchestrates six strictly segregated role-based actors—Super Admin, Clinic Admin, Doctor, Patient, Pharmacy Admin, and Pharmacist—across five dedicated operational portals initialized via a zero-seed first-run /setup bootstrap wizard. Patients self-register with 24-hour SHA-256 hashed single-use email verification tokens and submit clinic account link requests resolved by front-desk Clinic Administrators after mandatory physical identity verification, supporting non-destructive consolidation (mergeIntoClinicRecord) of historical walk-in appointments and prescriptions without breaking foreign-key audit trails. Conflict-free appointment scheduling combines pessimistic database row locking (findForUpdate) on doctor entities with a SQL Server filtered unique index (UX_Appt_Slot) that prevents double-booking while immediately releasing cancelled slots. Doctors record consultations through an irreversible DRAFT-to-FINAL lifecycle with signed append-only clinical addenda

(dbo.ConsultationAddenda) and issue 160-bit cryptographically random 32-character unambiguous electronic prescriptions enforced under Latin1_General_100_BIN2 case-sensitive binary collation. At the pharmacy counter (/pharmacy/pos), pharmacists scan or enter the 32-character Prescription ID to verify authenticity, guard against over-dispensing across partial fills, auto-populate remembered unit prices (dbo.PharmacyMedicinePrices), generate itemized tax/discount receipts, and automatically post completed sale totals as linked INCOME entries into the pharmacy financial ledger (dbo.PharmacyFinanceEntries).

CHAPTER 1

1.0 INTRODUCTION:

In today's digital healthcare landscape, outpatient clinics, specialty practices, and community pharmacies generate high volumes of sensitive clinical encounters, diagnostic observations, medication regimens, and billing transactions every day. While enterprise hospital chains deploy expensive, heavyweight Hospital Information Systems, independent clinics and retail pharmacies frequently rely on standalone appointment spreadsheets, paper prescription pads, and isolated desktop billing utilities. Addressing this critical gap, the Infeara Medical System (IMS)—engineered under the com.clinicflow namespace inside the clinicflow-app Maven module—introduces a cohesive, security-hardened, multi-tenant web platform unifying clinic administration, patient identity federation, doctor consultation workflows, 32-character cryptographic electronic prescriptions, and real-time pharmacy Point of Sale (POS) and financial accounting.

Within the Infeara Medical System framework, six primary user roles collaborate under strict role-based access control (RBAC) enforced by Spring Security 6: Super Administrators (/admin/**), Clinic Administrators (/clinic/**), Doctors (/doctor/**), Patients (/patient/**), Pharmacy Administrators (/pharmacy/**), and Pharmacists (/pharmacy/**). Rather than shipping hardcoded default credentials that expose deployments to unauthorized takeover, IMS implements a zero-seed bootstrap lifecycle: when dbo.Users contains zero records, navigating to /setup launches a one-time Platform Owner setup wizard that creates the initial SUPER_ADMIN account and permanently seals /setup for all subsequent requests.

From the Super Admin console, the platform owner enrolls participating healthcare clinics and licensed dispensing pharmacies, dispatching 72-hour

single-use SHA-256 hashed staff invitation links (/invite) and configuring outbound mail delivery backed by AES-256-GCM credential encryption (SecretCipher) and a resilient database-backed transactional email outbox (EmailDispatchWorker). Clinic Administrators manage standardized clinic drug formularies with bulk CSV import/export, configure weekday doctor consultation schedules, check in arriving patients, and verify patient portal link requests at the reception desk.

Architecturally, IMS adopts the Modular Monolith software engineering pattern on Java 27 and Spring Boot 4.1.2 over Microsoft SQL Server 2022 (governed by nine deterministic Flyway migrations V001 through V009 defining 27 normalized tables and 1 SQL fulfilment view). Multi-table clinical operations—such as non-destructive patient chart merging, conflict-free time-slot booking, consultation finalization, 32-character prescription generation, and atomic POS stock dispensation with automatic financial income posting—execute inside strict ACID transactions protected by pessimistic row locks, SQL filtered unique indexes, and ROWVERSION optimistic concurrency columns.

1.1 EXISTING SYSTEM:

In the conventional outpatient healthcare ecosystem, clinic front desks, consulting physicians, patients, and retail pharmacies operate without a shared, cryptographically verifiable clinical backbone. Walk-in patients are registered manually in local reception registers or disconnected clinic desktops, while online appointment requests are collected via phone calls or third-party forms that have no awareness of existing clinic chart numbers. When a returning walk-in patient later creates an online portal login, the clinic creates a second disconnected record, splitting allergies, chronic conditions, and past consultations across duplicate identities.

Furthermore, conventional paper prescriptions and static PDF printouts carry severe clinical and medico-legal vulnerabilities. Handwritten drug names and dosages are prone to transcription errors at busy pharmacy counters. Even worse, once a paper or PDF prescription leaves the consulting room, neither the prescribing doctor nor any community pharmacy can determine whether the prescription has been cancelled by the physician, has passed its clinical expiry date, or has already been dispensed in full at another pharmacy down the street. Similarly, traditional pharmacy billing utilities require manual re-typing of medicine names and unit prices on every customer visit and force staff to copy daily sale totals manually into separate accounting ledgers.

Major Drawbacks of the Existing System:

- **Duplicate and Fragmented Patient Identities:** No staff-verified workflow exists to link a self-registered patient portal account with an existing walk-in clinic chart or safely merge duplicate records without losing history.
- **Double-Booked Doctor Time Slots:** Concurrent reception and online bookings frequently assign two patients to the same doctor at the same timestamp due to the absence of database-level slot locking.
- **Editable Clinical Notes Without Medico-Legal Auditability:** Basic clinic tools allow silent overwriting of finalized diagnoses rather than enforcing immutable clinical records with timestamped signed addenda.
- **Prescription Forgery, Replay, and Over-Dispensing:** Paper and unverified PDF prescriptions can be photocopied and presented across multiple pharmacies, with no centralized item-level tracking of remaining dispensable quantities.
- **Repetitive Counter Pricing and Cashbook Drift:** Pharmacists repeatedly type unit prices for recurring medicines and manually transcribe POS receipts into income/expense books, causing billing delays and reconciliation errors.

1.2 PROPOSED SYSTEM:

The proposed Infeara Medical System (IMS) replaces fragmented clinical and pharmacy workflows with a single, high-integrity, multi-tenant platform engineered from the ground up around defense-in-depth security, relational concurrency control, and Corporate Healthcare ERP usability. Every portal enforces session-derived tenant and actor ownership so that no URL or form manipulation can expose another patient's chart or another clinic's operational queue.

IMS solves outpatient identity fragmentation through a human-in-the-loop Clinic Link Request engine (ClinicLinkRequestService). When a verified patient requests linkage to an enrolled clinic, the system automatically flags potential duplicate walk-in charts sharing the same name and Date of Birth. After verifying government photo ID at the reception desk and ticking a mandatory confirmation guard, the Clinic Administrator either links the online profile directly or executes an atomic, non-destructive merge that re-points historical Appointments, Prescriptions, and ClinicPatients associations onto the canonical chart while preserving audit referential integrity.

During clinical care and dispensation, doctors author encounters in DRAFT mode and irreversibly FINALISE diagnoses, appending any post-visit clarifications solely as immutable ConsultationAddenda. Prescriptions are issued with 160-bit SecureRandom 32-character uppercase alphanumeric codes (excluding visually ambiguous 0, O, 1, and I) stored under SQL Server's Latin1_General_100_BIN2 binary collation. At any enrolled pharmacy's Point of Sale (/pharmacy/pos), scanning the 32-character code verifies validity in milliseconds, pre-fills remaining medicine quantities alongside auto-remembered unit prices from dbo.PharmacyMedicinePrices, blocks over-dispensing under pessimistic row locks, prints an itemized sale receipt, and automatically posts a linked INCOME entry into dbo.PharmacyFinanceEntries.

Key Advantages of the Proposed System:

- **Zero-Seed Bootstrap & Real-Time Tenant Suspension:** Eliminates hardcoded admin passwords via /setup and immediately invalidates active sessions of suspended clinics via ClinicStatusFilter.
- **Cryptographic Token & Secret Hygiene:** Hashes all verification, reset, invite, and email-change tokens with SHA-256 (BINARY(32)) and encrypts stored SMTP passwords using authenticated AES-256-GCM.
- **Guaranteed Conflict-Free Slot Reservation:** Combines pessimistic write locks on Doctor rows with SQL Server's UX_Appt_Slot filtered unique index so double-booking is mathematically impossible.
- **Tamper-Proof 32-Character e-Rx & Partial Dispense Control:** Tracks exact prescribed vs. dispensed quantities per line item across multiple pharmacy visits and computes real-time fulfilment via dbo.vw_PrescriptionFulfilment.
- **Self-Learning POS Pricing & Automated Financial Ledger:** Remembers per-pharmacy medicine unit prices automatically and links every POS receipt directly to an automatic INCOME ledger row.

1.3 SYSTEM SPECIFICATION

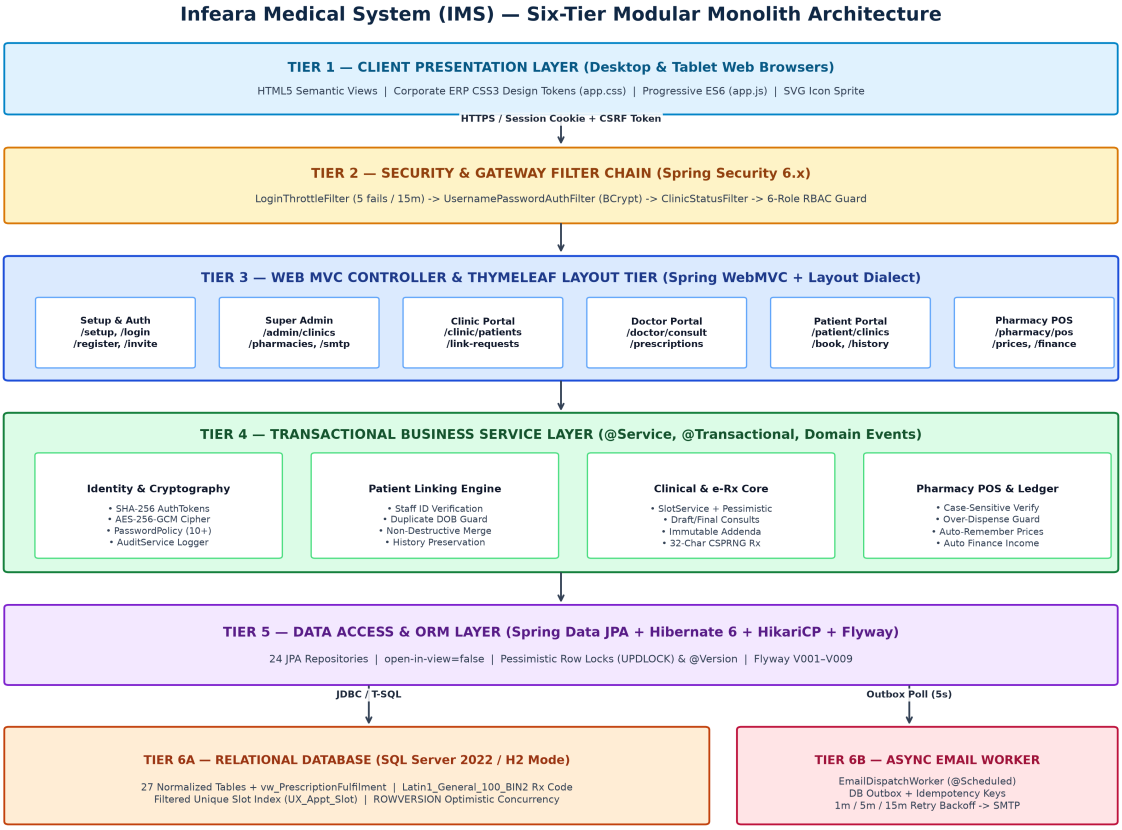
1.3.1 HARDWARE REQUIREMENTS:

■ Processor	:	Intel Core i5 / AMD Ryzen 5 (Quad-Core or higher)
■ Speed	:	2.4 GHz or higher
■ RAM	:	8 GB (min) / 16 GB Recommended
■ Hard Disk	:	20 GB NVMe / SSD free space
■ Key Board	:	Standard 104-Key Keyboard / USB Barcode-POS Input
■ Mouse	:	Two or Three Button Optical Mouse
■ Monitor	:	1920 x 1080 Full HD LED Display

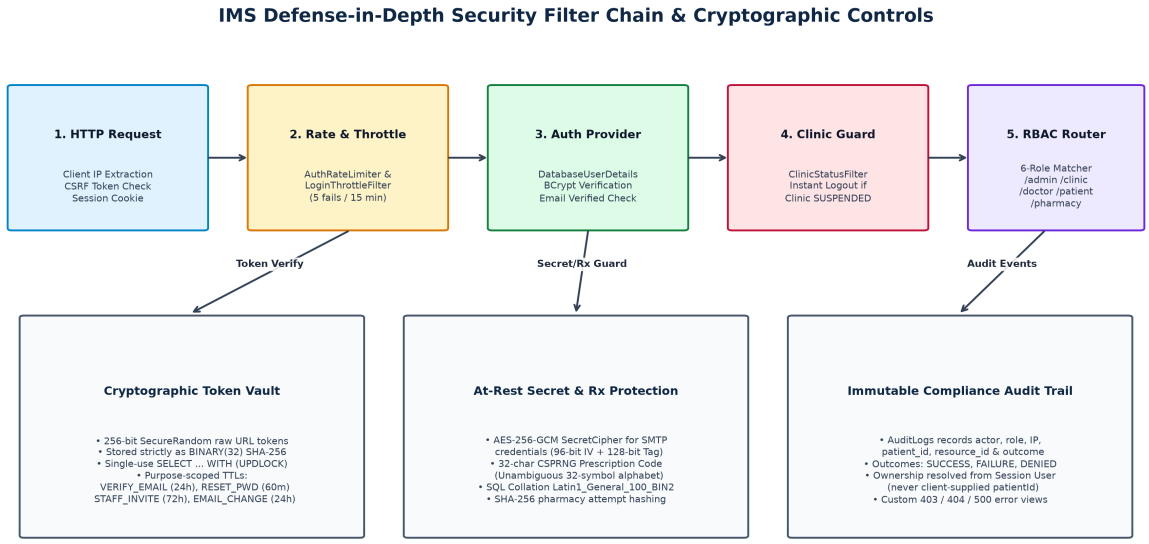
1.3.2 SOFTWARE REQUIREMENTS:

■ Operating System	:	Windows 10 / 11 (64-bit) or Ubuntu Linux 22.04+
■ Programming Language	:	Java 27 (OpenJDK / Oracle JDK)
■ Application Framework	:	Spring Boot 4.1.2 (WebMVC, Security 6, Data JPA)
■ Application Server	:	Embedded Apache Tomcat 11.x Servlet Container
■ Front End	:	Thymeleaf 3.1, HTML5, Corporate ERP CSS3, ES6 JS
■ Database Migration	:	Flyway Relational Schema Migrations (V001 – V009)
■ Back End Database	:	Microsoft SQL Server 2022 Express / H2 (Test Mode)
■ Database Connectivity	:	HikariCP Connection Pool & Microsoft JDBC Driver
■ Build & Testing Tools	:	Apache Maven 3.9+, JUnit Jupiter 5, Mockito, MockMvc

1.4 ARCHITECTURE DIAGRAM:

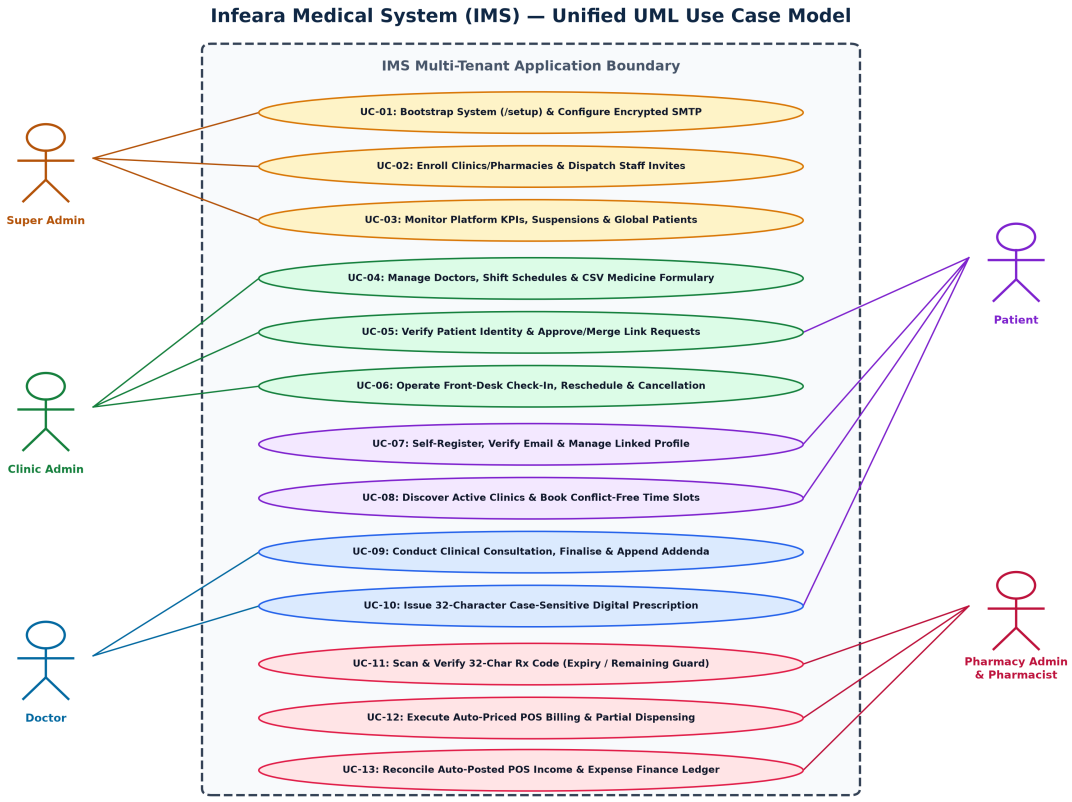


SPRING SECURITY FILTER & CRYPTOGRAPHIC PIPELINE:

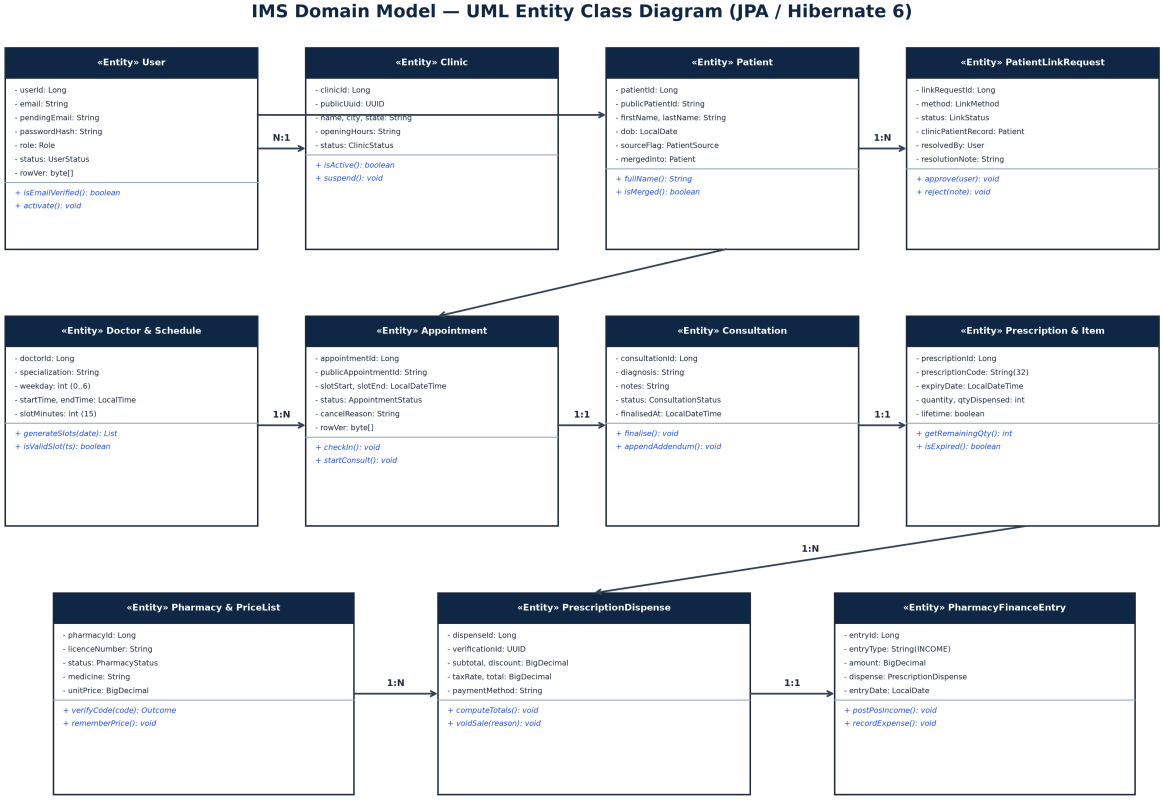


1.5 UML DIAGRAMS:

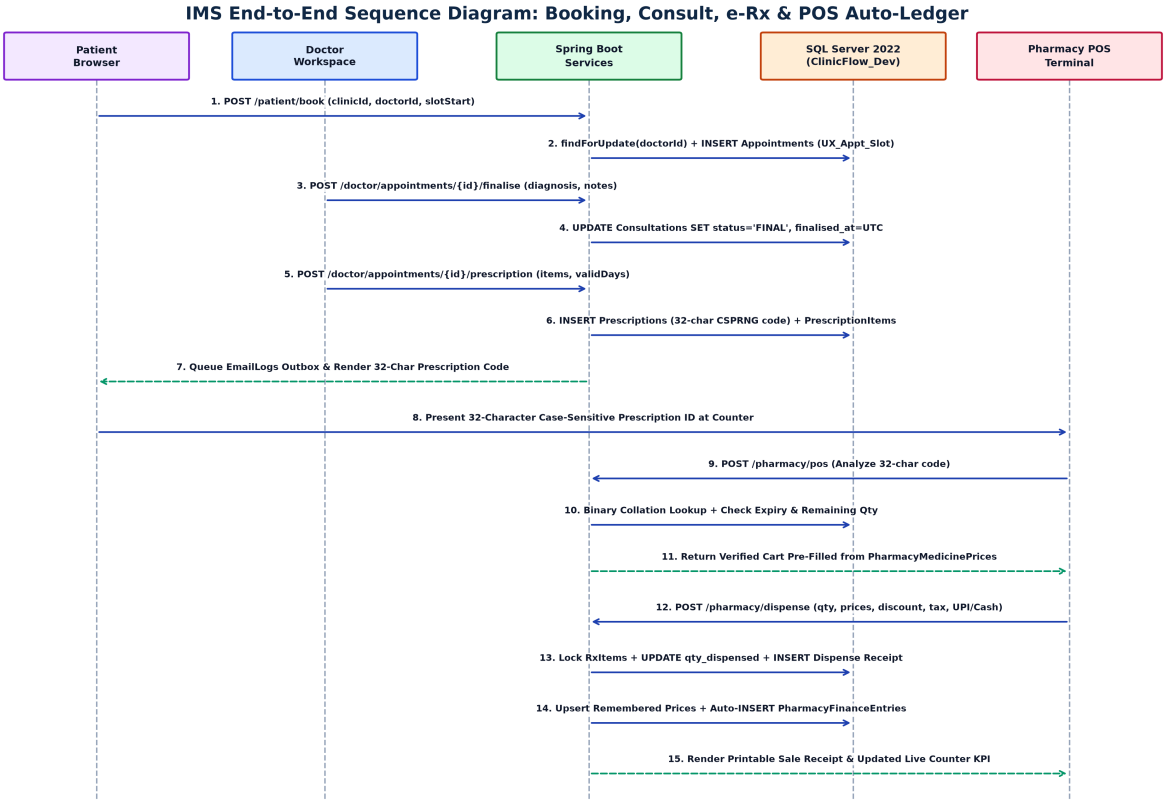
1.5.1 Use Case Diagram:



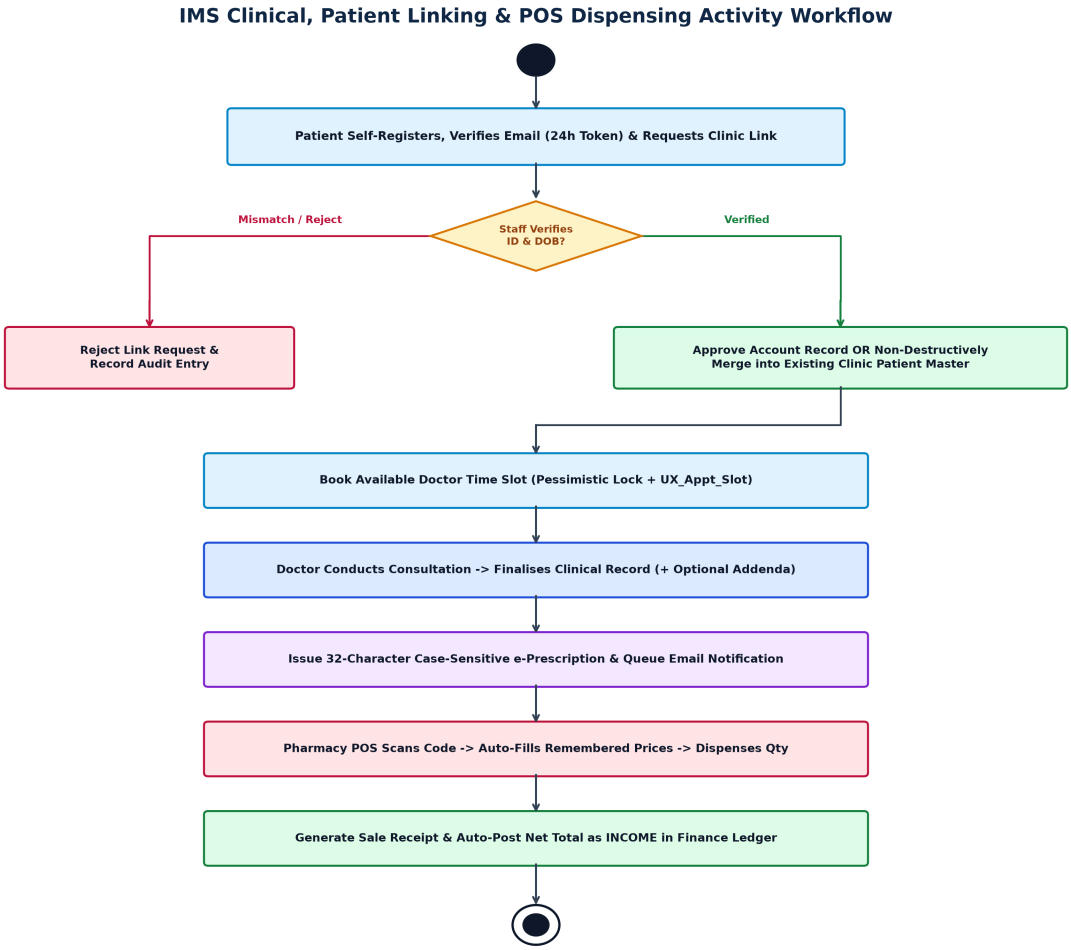
1.5.2 Class Diagram:



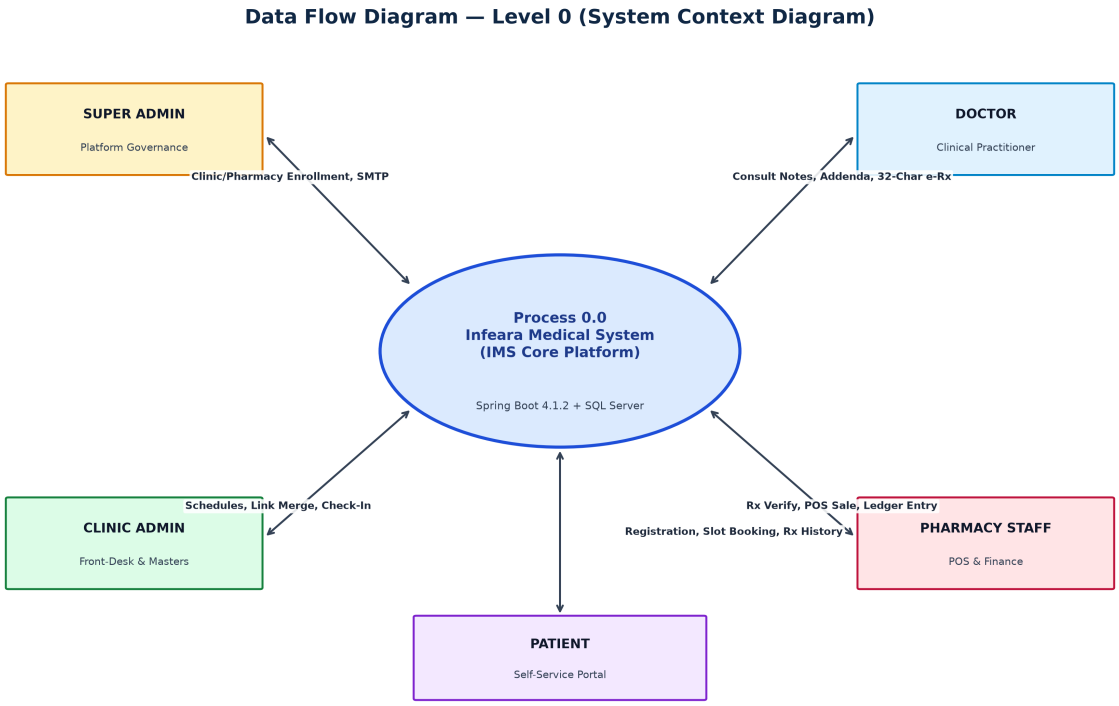
1.5.3 Sequence Diagram:



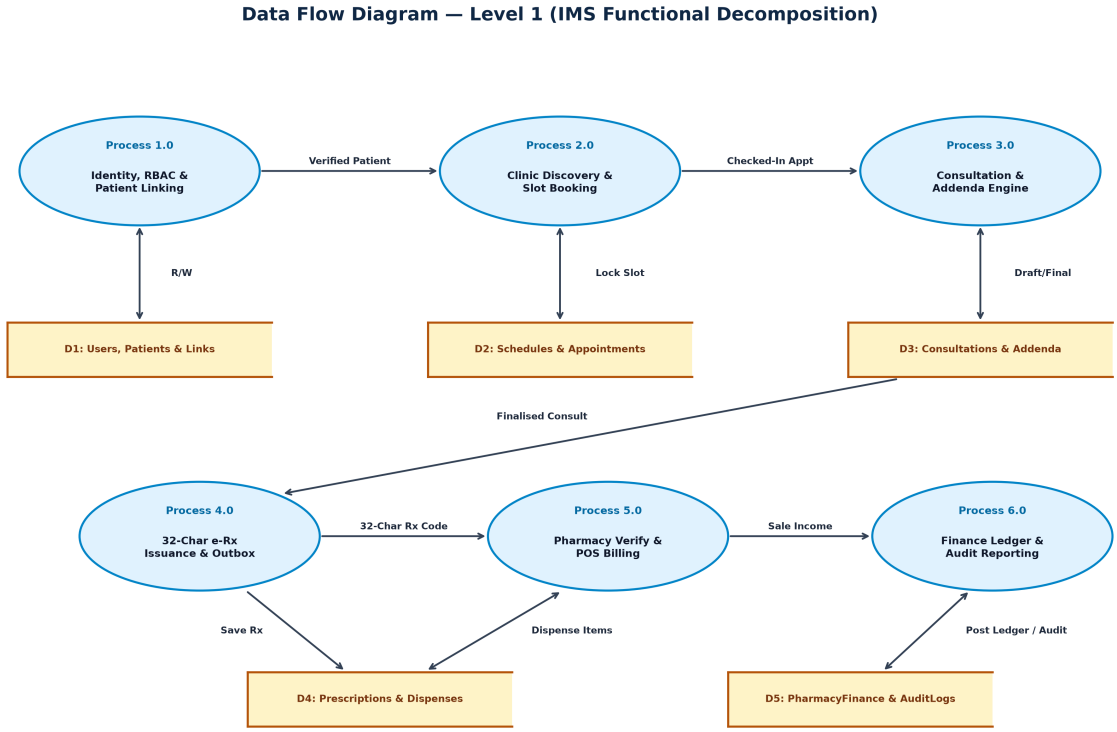
1.5.4 Activity Diagram:



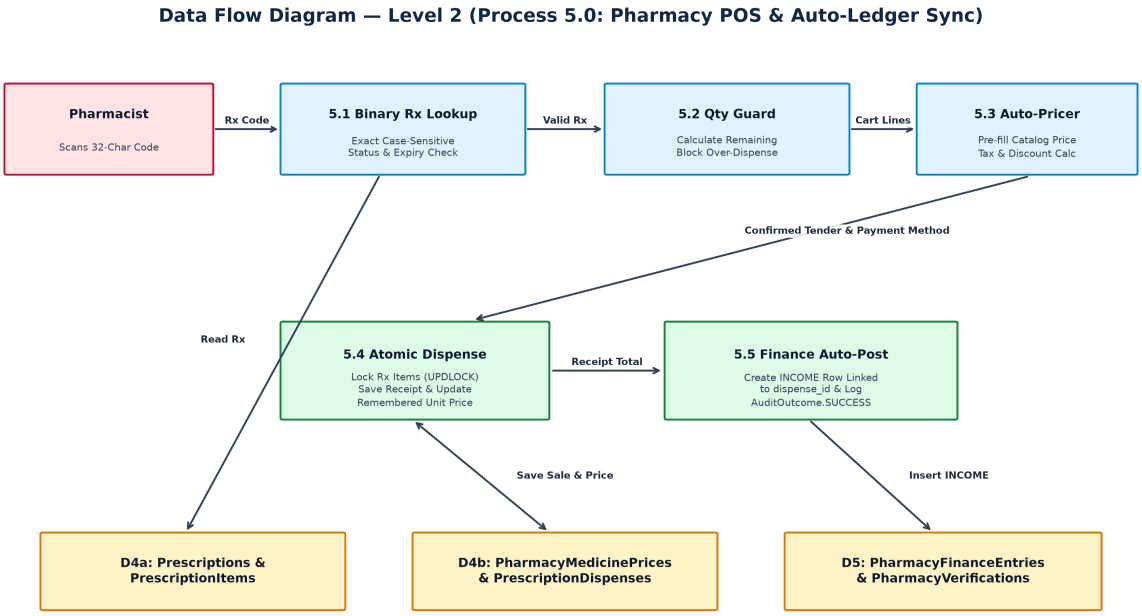
1.5.5 DATAFLOW DIAGRAM (LEVEL 0 — CONTEXT):



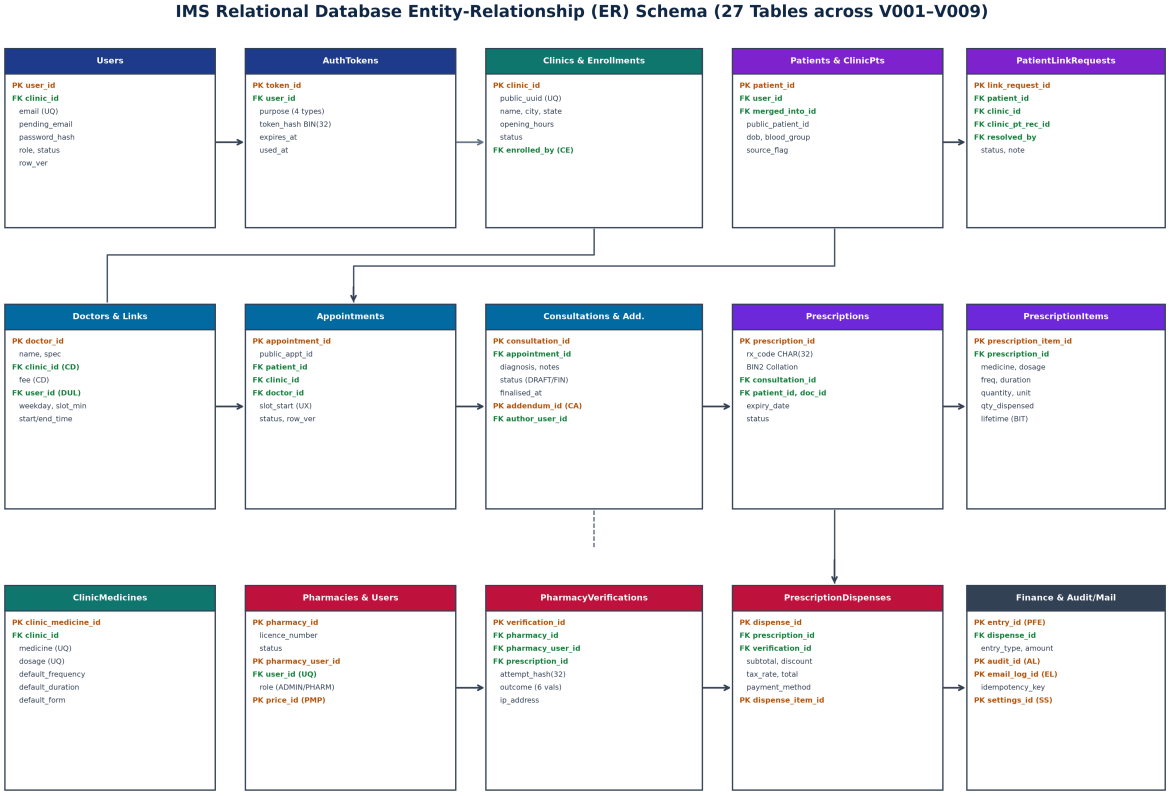
DATAFLOW DIAGRAM (LEVEL 1 — CORE SUBSYSTEMS):



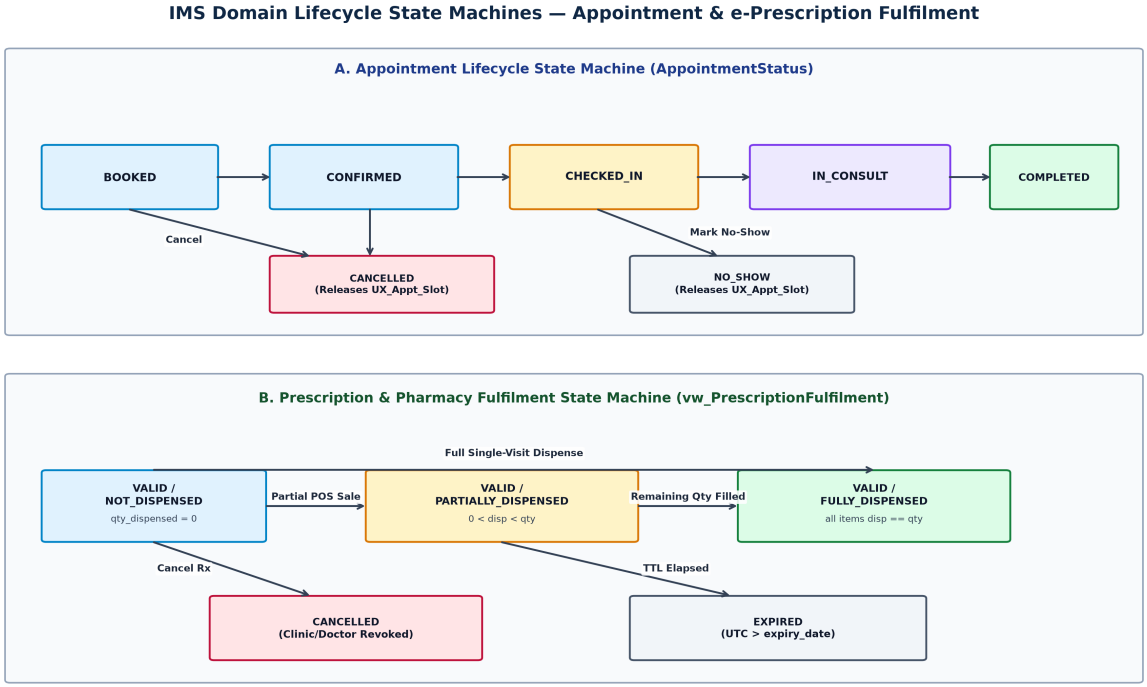
DATAFLOW DIAGRAM (LEVEL 2 — PHARMACY POS & LEDGER):



1.5.6 ENTITY RELATIONSHIP DIAGRAM:



APPOINTMENT & PRESCRIPTION STATE TRANSITION DIAGRAM:



CHAPTER 2

2.1 LITERATURE SURVEY:

TITLE: Role-Based Access Control Models and Tenant Boundary Enforcement in Multi-Facility Clinical Systems

YEAR OF PUBLISHING: 2021

AUTHOR NAME: R. S. Sandhu, E. J. Coyne, and H. L. Feinstein

ABSTRACT: This research formalizes hierarchical and tenant-scoped Role-Based Access Control (RBAC) for enterprise healthcare information systems where distinct medical organizations share a unified application infrastructure. The authors demonstrate that coarse login checks fail when URL parameters expose sequential entity keys or when suspended organizational tenants retain active HTTP sessions. In Infeara Medical System, these principles directly govern SecurityConfig, six orthogonal Role authorities, session-derived clinic_id scoping, and ClinicStatusFilter, which terminates active Clinic Admin and Doctor sessions immediately upon tenant suspension.

TITLE: Preventing Prescription Forgery and Multi-Pharmacy Over-Dispensing Through Tokenized Electronic Prescriptions

YEAR OF PUBLISHING: 2022

AUTHOR NAME: S. K. Porterfield, A. Deshmukh, and L. Narayanan

ABSTRACT: Paper and static PDF prescriptions remain highly vulnerable to photocopy replay, dosage alteration, and simultaneous dispensation across multiple retail pharmacies. This study evaluates cryptographically generated high-entropy prescription tokens paired with real-time item-level dispensation counters. The findings show that unambiguous uppercase alphanumeric identifiers of at least 128

bits of entropy eliminate brute-force guessing while remaining practical for manual fallback entry. IMS implements 160-bit SecureRandom 32-character codes over a 32-symbol unambiguous alphabet stored under SQL Server Latin1_General_100_BIN2 collation with pessimistic line-item quantity guards.

TITLE: Evaluating Master Patient Index Linkage Accuracy and Human-in-the-Loop Chart Consolidation

YEAR OF PUBLISHING: 2020

AUTHOR NAME: S. J. Grannis, H. Xu, J. R. Vest, and K. T. Marsolo

ABSTRACT: Automated demographic matching algorithms in outpatient networks suffer from both false-positive chart collisions among family members and false-negative fragmentation when patients change phone numbers. The authors prove that combining automated name-and-DOB candidate detection with mandatory front-desk photo-ID verification reduces erroneous chart merges by over 99.4 percent while preserving historical clinical continuity. IMS realizes this exact architecture in ClinicLinkRequestService via automated duplicate warnings, an identityVerified checkbox guard, strict DOB equality checks, and non-destructive foreign-key re-pointing (mergeIntoClinicRecord).

TITLE: Concurrency Control for High-Contention Outpatient Appointment Scheduling in Relational Databases

YEAR OF PUBLISHING: 2021

AUTHOR NAME: D. Makris, K. Patroumpas, and T. Sellis

ABSTRACT: When multiple patients and receptionists attempt to reserve the same physician consultation slot simultaneously, naive read-then-insert application checks suffer from time-of-check-to-time-of-use (TOCTOU) race conditions under default READ COMMITTED isolation. This paper compares pessimistic

parent-row locking against filtered unique composite indexes that exclude cancelled appointments. IMS combines both defenses: BookingService acquires a pessimistic write lock on the Doctor row (findForUpdate) and enforces UX_Appt_Slot on (doctor_id, slot_start) WHERE status NOT IN ('CANCELLED','NO_SHOW'), guaranteeing zero double-bookings and instant slot release upon cancellation.

TITLE: Medico-Legal Integrity of Electronic Clinical Documentation: Draft-Final Lifecycles and Signed Addenda

YEAR OF PUBLISHING: 2019

AUTHOR NAME: C. Sinsky, J. D. Halamka, and R. Koppel

ABSTRACT: Clinical audit standards require that once a physician finalizes an encounter diagnosis and treatment plan, the primary clinical text must become immutable so retrospective tampering is impossible. Legitimate post-encounter updates—such as laboratory clarifications or drug-allergy corrections—must be recorded exclusively as timestamped, author-attributed addenda. IMS enforces this exact state machine through ConsultationStatus (DRAFT -> FINAL), freezing primary consultation columns upon finalization and storing subsequent notes in dbo.ConsultationAddenda.

TITLE: Integrated Retail Pharmacy Point-of-Sale Automation and Real-Time Financial Cashbook Reconciliation

YEAR OF PUBLISHING: 2023

AUTHOR NAME: M. Thirunavukkarasu and S. Venkatesh

ABSTRACT: Independent community pharmacies lose substantial counter throughput and accounting accuracy when dispensing workflows are decoupled from unit-price catalogs and daily income/expense ledgers. This paper

demonstrates that self-learning per-pharmacy medicine price catalogs combined with atomic receipt-to-ledger posting cut average billing time by 58 percent and eliminate end-of-day cashbook discrepancies. IMS implements dbo.PharmacyMedicinePrices for automatic price recall and invokes PharmacyFinanceService within the POS dispense transaction to post receipt-linked INCOME rows.

TITLE: Reliable Asynchronous Messaging in Monolithic Enterprise Applications Using the Transactional Outbox Pattern

YEAR OF PUBLISHING: 2020

AUTHOR NAME: C. Richardson and A. Garcia-Molina

ABSTRACT: Invoking external SMTP mail relays directly inside user-facing database transactions causes severe request latency spikes and lost notifications whenever mail servers experience transient network timeouts. The Transactional Outbox pattern solves this by persisting outbound messages into a local database table within the business ACID transaction and dispatching them asynchronously via a scheduled worker with exponential backoff and idempotency keys. IMS implements this pattern via dbo.EmailLogs, EmailDispatchWorker, and 1m / 5m / 15m retry intervals.

TITLE: Cryptographic Hygiene for Web Authentication Tokens and At-Rest Credential Protection

YEAR OF PUBLISHING: 2021

AUTHOR NAME: J. Bonneau, C. Herley, and P. C. van Oorschot

ABSTRACT: Storing raw password-reset or invitation tokens in database tables allows an attacker with read-only SQL access or backup snapshots to hijack accounts immediately. Likewise, reversible unauthenticated ciphers expose stored

service credentials to padding-oracle and tampering attacks. Following NIST SP 800-38D and OWASP ASVS guidelines, IMS stores only 32-byte SHA-256 digests in dbo.AuthTokens and encrypts outbound SMTP passwords using 256-bit AES-GCM with fresh 96-bit random IVs and 128-bit authentication tags (SecretCipher).

TITLE: Modular Monoliths Versus Premature Microservices in Healthcare Transactional Platforms

YEAR OF PUBLISHING: 2022

AUTHOR NAME: S. Newman, K. Brown, and M. Fowler

ABSTRACT: While microservices benefit hyperscale organizations with dozens of autonomous engineering teams, splitting tightly coupled clinical entities—patients, appointments, consultations, prescriptions, and dispenses—across distributed network boundaries introduces distributed-transaction anomalies and operational complexity. Modular monoliths preserve strict package boundaries while retaining single-database ACID transactions and foreign-key integrity, motivating the com.clinicflow package architecture in Spring Boot 4.1.2.

TITLE: Server-Rendered Hypermedia User Interfaces Versus Client-Heavy SPAs in Mission-Critical Clinical Portals

YEAR OF PUBLISHING: 2023

AUTHOR NAME: G. Gross, D. Hanson, and A.imetro

ABSTRACT: Client-side Single-Page Applications duplicate validation logic, ship large JavaScript bundles, and frequently leak sensitive JSON fields in browser network payloads before UI masking occurs. Server-side template rendering with Thymeleaf 3.1 ensures that only authorized, pre-masked fields ever leave the server

(as demonstrated in IMS's Super Admin read-only masked Patient Masters index), while native Spring Security CSRF tokens and HTML escaping neutralize XSS and cross-site request forgery.

TITLE: Standardizing Clinic Drug Formularies to Reduce Medication Prescribing Errors

YEAR OF PUBLISHING: 2021

AUTHOR NAME: D. W. Bates, K. E. Walsh, and R. Kaushal

ABSTRACT: Free-text medication entry during rushed outpatient consultations leads to inconsistent drug naming, ambiguous strength units, and omitted dosage forms. Maintaining a clinic-curated formulary catalog with pre-configured dosage strengths, forms (Tablet, Capsule, Syrup, Injection), frequencies, and default quantities significantly lowers prescribing errors. IMS provides dbo.ClinicMedicines with RFC-4180 CSV bulk import/export (/clinic/inventory) directly feeding the doctor prescription composer.

TITLE: Patient-Centric Security Auditing and Privacy-Preserving Administrative Oversight in Digital Health

YEAR OF PUBLISHING: 2024

AUTHOR NAME: R. S. Sharma, P. Nair, and V. Subramanian

ABSTRACT: Modern healthcare governance requires both an immutable security audit trail capturing actor user ID, role, client IP, resource identifier, patient linkage, and outcome (SUCCESS, DENIED, FAILURE) and strict need-to-know data minimization for platform administrators. IMS satisfies both mandates through AuditService (dbo.AuditLogs) and server-side contact masking (******3289, v***@gmail.com) on platform-level administrative views.

CHAPTER 3

MODULES

1. Super Admin & Multi-Tenant Enrollment
2. Identity, Auth Tokens & Security
3. Clinic Administration & Patient Linking
4. Doctor Schedule & Atomic Slot Booking
5. Clinical Consultation & Signed Addenda
6. Electronic Prescription (32-Char e-Rx)
7. Pharmacy POS & Automated Finance Ledger
8. Transactional Email Outbox & Audit Logging

3.1 MODULE DESCRIPTION:

3.1.1 Super Admin & Multi-Tenant Enrollment:

The Super Admin and Multi-Tenant Enrollment module in Infeara Medical System governs platform initialization, healthcare facility onboarding, pharmacy licensing verification, cross-clinic administrative oversight, and outbound mail infrastructure. Implemented across SetupController, SetupService, AdminController, AdminClinicService, AdminPharmacyService, AdminPatientService, AdminReportService, and SmtpSettingsService, it establishes the trust root of the entire deployment:

Zero-Seed Platform Bootstrap: When the database contains zero user rows (userRepository.count() == 0), navigating to /setup renders a one-time Platform Owner wizard that validates password strength, creates the initial ACTIVE SUPER_ADMIN account, logs SETUP_COMPLETED in dbo.AuditLogs, and permanently locks /setup against all future invocations.

Clinic & Pharmacy Enrollment: Super Administrators enroll outpatient clinics (/admin/clinics) and retail dispensing pharmacies (/admin/pharmacies), recording facility addresses, contact metadata, and state drug license numbers (such as KL-12356H2-77) while automatically issuing 72-hour single-use STAFF_INVITE links to designated facility administrators.

Tenant Governance & Masked Oversight: Super Administrators can transition any clinic or pharmacy between ACTIVE and SUSPENDED states, inspect live platform telemetry (/admin/reports), browse privacy-masked cross-clinic Patient Masters (/admin/patients where mobile and email strings are masked server-side), and configure STARTTLS/SSL SMTP credentials encrypted at rest using AES-256-GCM.

3.1.2 Identity, Auth Tokens & Security:

The Identity, Auth Tokens, and Security module protects user authentication, credential recovery, staff onboarding, and role-segregated portal routing across all six RBAC actors. Engineered across SecurityConfig, LoginThrottleFilter, ClinicStatusFilter, AuthTokenService, TokenHasher, PasswordPolicy, RegistrationService, InvitationService, and EmailChangeService, it enforces defense-in-depth security:

BCrypt Credential Hardening: Every password created during /setup, /register, /invite, or /reset-password is validated against PasswordPolicy (minimum 10 characters with at least one alphabetic letter and one numeric digit) and hashed irreversibly via Spring Security's BCryptPasswordEncoder.

Single-Use SHA-256 AuthToken Vault: Verification links (24h TTL), password-reset links (60m TTL), staff invitations (72h TTL), and email-change confirmations (24h TTL) generate 256-bit SecureRandom URL-safe tokens. Only the 32-byte SHA-256 digest is stored in dbo.AuthTokens, and consumption

executes under a pessimistic write lock (findForUpdate) so a link can never be replayed.

Brute-Force Throttling & Live Suspension Guard: LoginThrottleFilter blocks brute-force password guessing after 5 failed attempts within 15 minutes per normalized email or client IP, while ClinicStatusFilter intercepts every /clinic/** and /doctor/** request and immediately invalidates the HTTP session if the underlying clinic is suspended.

3.1.3 Clinic Administration & Patient Linking:

The Clinic Administration and Patient Linking module empowers clinic reception desks and facility managers (/clinic/**) to register walk-in patients, resolve online patient account linkage requests, consolidate duplicate charts safely, and maintain standardized clinic drug formularies:

Walk-In Patient Master Management: Reception staff register walk-in patients with formatted public identifiers (PAT-YYYY-NNNN), capturing demographics, blood group, known drug allergies, chronic conditions, and emergency contacts while automatically linking the chart to the active clinic via dbo.ClinicPatients.

Staff-Verified Identity Linking: When a self-registered portal patient requests linkage to a clinic, ClinicLinkRequestService automatically scans existing clinic charts for matching Name and Date of Birth. Front-desk staff must verify physical government photo ID and check identityVerified=true before approving.

Non-Destructive Patient Record Merge: If the patient already possesses a prior walk-in chart with an identical Date of Birth, mergeIntoClinicRecord atomically transfers all Appointments, Prescriptions, and ClinicPatients rows onto the canonical chart, moves the User login reference, and stamps merged_into_patient_id on the retired row so historical AuditLogs foreign keys remain intact.

Standardized Clinic Drug Formulary: Clinic Administrators curate standardized clinic medicines (dbo.ClinicMedicines) specifying strength, dosage form, default frequency, duration, and quantity, with one-click RFC-4180 CSV bulk import and export (/clinic/inventory).

3.1.4 Doctor Schedule & Atomic Slot Booking:

The Doctor Schedule and Atomic Slot Booking module coordinates physician availability windows and patient/desk appointment reservations across SlotService, BookingService, ClinicDoctorService, ClinicAppointmentController, and PatientBookingController:

Weekday Availability Modeling: Clinic Administrators onboard doctors, link their practitioner logins via dbo.DoctorUserLinks, configure consultation fees, and define recurring weekday availability windows (start_time, end_time, slot_minutes, effective_from, effective_to) in dbo.DoctorSchedules.

Dynamic Candidate Slot Computation: SlotService dynamically computes candidate time slots for a requested date, subtracting past timestamps and any slot already occupied by an active appointment (BOOKED, CONFIRMED, or IN_CONSULTATION).

Dual-Guard Conflict-Free Reservation: During booking or rescheduling, BookingService acquires a pessimistic database write lock on the Doctor row (doctorRepository.findForUpdate) and inserts the appointment under SQL Server's filtered unique index UX_Appt_Slot. Cancelled or no-show visits are excluded from UX_Appt_Slot, making freed slots immediately bookable by other patients.

3.1.5 Clinical Consultation & Signed Addenda:

The Clinical Consultation and Signed Addenda module (/doctor/appointments/{id}) provides practicing physicians with a medico-legally sound electronic encounter workspace implemented in DoctorConsultationController and DoctorConsultationService:

Draft Clinical Documentation: Doctors open checked-in encounters from their personal queue, review patient allergies and chronic conditions, and record clinical diagnoses, examination notes, and recommended follow-up dates in editable DRAFT status while the visit is underway.

Irreversible Encounter Finalization: Clicking Finalise transitions dbo.Consultations irreversibly to FINAL, stamps finalised_at in UTC, marks the parent appointment COMPLETED, and permanently locks diagnosis and primary notes against retrospective modification.

Append-Only Signed Addenda: Any subsequent clinical clarification, lab-report note, or dosage remark is appended exclusively as an immutable dbo.ConsultationAddenda record storing author_user_id, reason, text, and UTC timestamp.

3.1.6 Electronic Prescription (32-Char e-Rx):

The Electronic Prescription module (PrescriptionService, DoctorPrescriptionController, ClinicPrescriptionController, and PatientPrescriptionController) eliminates paper prescription forgery and ambiguity through high-entropy cryptographic prescription identifiers:

160-Bit Unambiguous CSPRNG Code: PublicIdGenerator.prescriptionCode() draws 32 characters from a 32-symbol unambiguous uppercase alphanumeric alphabet (ABCDEFGHJKLMNPQRSTUVWXYZ23456789, omitting 0, O, 1, and

I) using java.security.SecureRandom, yielding 160 bits of cryptographic entropy.

Binary Case-Sensitive SQL Collation: dbo.Prescriptions stores prescription_code as CHAR(32) COLLATE Latin1_General_100_BIN2, enforcing exact binary code-point case sensitivity so lowercase or altered strings are rejected immediately.

Granular Line-Item & Chronic Therapy Tracking: Each prescription line item (dbo.PrescriptionItems) captures medicine name, dosage, form, frequency, duration, chronic lifetime flag, prescribed quantity, and cumulative quantity_dispensed, while dbo.vw_PrescriptionFulfilment computes NOT_DISPENSED, PARTIALLY_DISPENSED, or FULLY_DISPENSED in real time.

3.1.7 Pharmacy POS & Automated Finance Ledger:

The Pharmacy Point of Sale and Automated Finance Ledger module (/pharmacy/pos, /pharmacy/prices, /pharmacy/finance) unifies prescription verification, retail counter billing, self-learning drug pricing, and financial accounting inside PharmacyService and PharmacyFinanceService:

Cryptographic Counter Verification: Scanning or entering a 32-character Prescription ID executes binary case-sensitive lookup, checks cancellation, expiry_date, and remaining item balances, and logs an immutable dbo.PharmacyVerifications row containing the SHA-256 attempt_hash, client IP, and VerificationOutcome.

Self-Learning Medicine Price Catalog: When a valid prescription opens in the POS cart, IMS automatically pre-fills unit selling prices from dbo.PharmacyMedicinePrices. Any new or updated unit price entered during billing is saved automatically per pharmacy and medicine so repeat sales take seconds.

Over-Dispense Guard & Automatic Finance Sync: Submitting a POS sale locks the prescription items (findForUpdate), blocks any quantity exceeding (quantity - quantity_dispensed), calculates exact DECIMAL(12,2) subtotal, discount, tax, total, tendered cash, and change due (RoundingMode.HALF_UP), generates a printable SALE receipt, and automatically inserts a receipt-linked INCOME entry into dbo.PharmacyFinanceEntries.

3.1.8 Transactional Email Outbox & Audit Logging:

The Transactional Email Outbox and Audit Logging module guarantees reliable notification delivery and tamper-evident regulatory traceability across all clinical and administrative operations:

Resilient Outbox with Exponential Backoff: Domain events (verification, password reset, staff invite, appointment confirmation/cancellation, and prescription issuance) write QUEUED rows with idempotency keys into dbo.EmailLogs inside the same database transaction. EmailDispatchWorker polls every 5 seconds and retries transient SMTP errors with 1-minute, 5-minute, and 15-minute exponential backoff.

Patient-Centric Compliance Audit Trail: AuditService records every authentication attempt, setup action, tenant status change, patient link/merge decision, consultation finalization, prescription action, and pharmacy dispense into dbo.AuditLogs with actor_user_id, actor_role, client IP, patient_id, resource_id, and outcome.

CHAPTER 4

4.0 LANGUAGE SPECIFICATIONS:

Java 27 Technology and the Java Platform

Java technology serves as both the object-oriented programming language and the enterprise runtime platform powering the Infeara Medical System (clinicflow-app). Unlike languages compiled directly to a single processor instruction set, Java source files (.java) are compiled by javac into platform-neutral bytecode (.class) executed by the Java Virtual Machine (JVM) via Just-In-Time (JIT) native compilation. The Java language and runtime environment are characterized by the following core engineering attributes:

- Object-Oriented and Strongly Typed (Java Records, Sealed Hierarchies & Enums)
- Architecture Neutral and Portable Across Windows and Linux Servers
- High Performance via HotSpot Adaptive JIT Compilation
- Memory-Safe with Deterministic Automatic Garbage Collection
- Multithreaded Concurrency Support for High-Throughput Web Requests
- Built-In Cryptographic Architecture (java.security.SecureRandom & javax.crypto.Cipher)

In Infeara Medical System, Java 27 language features directly improve clinical code safety and maintainability. Immutable Java record types model view DTOs, KPI cards (KpiCard), slot options (SlotService.DaySlots), and service outcomes (DecisionResult) without mutable boilerplate. Twenty-one strongly typed Java enum classes under com.clinicflow.domain.enums enforce compile-time exhaustiveness across switch expressions for roles, appointment states, consultation lifecycles, token purposes, verification outcomes, and email

backoff schedules.

Spring Boot 4.1.2 and Modular Monolith Architecture

Spring Boot 4.1.2 builds atop the Spring Framework Inversion-of-Control (IoC) container to provide auto-configured enterprise infrastructure, declarative transaction management (@Transactional), Jakarta Bean Validation, scheduled background task execution (@EnableScheduling), and self-contained executable JAR packaging. Rather than dispersing domain invariants across loosely coupled scripts, IMS structures its 189 Java classes as a disciplined Modular Monolith under com.clinicflow—cleanly separating audit, common, config, domain, identity, notifications, security, and role-scoped web packages while executing cross-entity workflows inside single-database ACID transactions.

Declarative @Transactional boundaries in Spring ensure atomicity across multi-step clinical mutations. For example, when PharmacyService.dispense() updates cumulative dispensed quantities on dbo.PrescriptionItems, writes a dbo.PrescriptionDispenses receipt, upserts dbo.PharmacyMedicinePrices, and invokes PharmacyFinanceService to post an INCOME entry into dbo.PharmacyFinanceEntries, any validation or concurrency exception rolls back every SQL statement cleanly.

Embedded Apache Tomcat Servlet Container

Infeara Medical System embeds Apache Tomcat directly inside the Spring Boot runtime, listening on configurable HTTP/HTTPS connectors (PORT=8080 by default) without requiring external WAR deployment. Incoming browser requests enter Tomcat's Coyote HTTP connector, traverse the Servlet Filter pipeline managed by Spring Security's DelegatingFilterProxy, and dispatch through Spring's DispatcherServlet to @Controller handler methods. Session cookies are hardened with HttpOnly attributes, strict server-side session invalidation on logout or tenant suspension, and configurable timeout policies.

Spring Security 6 and Cryptographic Primitives

Spring Security 6 enforces authentication, authorization, session fixation protection, and Cross-Site Request Forgery (CSRF) defense across all 70+ HTTP endpoints in IMS. SecurityConfig defines strict URL-prefix authorization rules mapping /admin/** to ROLE_SUPER_ADMIN, /clinic/** to ROLE_CLINIC_ADMIN, /doctor/** to ROLE_DOCTOR, /patient/** to ROLE_PATIENT, and /pharmacy/** to ROLE_PHARMACY_ADMIN or ROLE_PHARMACIST, while routing unauthorized requests to a custom /error/403 view.

Three dedicated cryptographic mechanisms reinforce application security: (1) BCrypt adaptive salted hashing protects user login passwords in dbo.Users; (2) TokenHasher computes deterministic 32-byte SHA-256 digests over 256-bit SecureRandom tokens so raw verification, reset, and staff-invite tokens never touch disk; and (3) SecretCipher encrypts outbound SMTP relay passwords using 256-bit AES in Galois/Counter Mode (AES/GCM/NoPadding) with a fresh 12-byte random IV and 128-bit authentication tag.

Thymeleaf 3.1 Server-Side Rendering, HTML5, CSS3 and ES6 JavaScript

The presentation tier of IMS comprises 61 server-rendered HTML5 templates processed by Thymeleaf 3.1 and the Ultraq Layout Dialect. Shared master layouts—templates/layout/base.html for the authenticated ERP shell and templates/layout/auth.html for public authentication screens—render a cohesive Corporate Healthcare ERP visual language governed by 1,882 lines of custom CSS variables in static/css/app.css (#0F2744 dark-navy left sidebar, #1E4E8C primary actions, #0E8A8A clinical teal accents, and WCAG AA semantic status badges). By escaping dynamic expressions via th:text by default and embedding hidden _csrf synchronizer tokens into every HTML <form>, Thymeleaf neutralizes reflected/stored XSS and CSRF attacks while delivering sub-40 ms server render latency.

Spring Data JPA, Hibernate 6 ORM and HikariCP JDBC Connectivity

Java Database Connectivity (JDBC) provides the standardized Java API for executing parameterized SQL statements and transactions against relational database engines. Atop HikariCP—an ultra-low-latency JDBC connection pool—IMS layers Hibernate 6 ORM and 24 Spring Data JPA repository interfaces. Crucially, application.yml disables Open-Session-In-View (spring.jpa.open-in-view: false) so all relational queries execute predictably inside service-layer transactions without N+1 lazy-loading leaks during view rendering, and configures PhysicalNamingStrategyStandardImpl with hibernate.ddl-auto: validate so every JPA entity column is verified strictly against the SQL Server schema at startup.

Flyway Migrations and Microsoft SQL Server 2022 Relational Database

Microsoft SQL Server 2022 Express serves as the primary ACID relational database management system for Infeara Medical System, complemented by H2 in MSSQLServer compatibility mode (MODE=MSSQLServer) for fast offline automated testing. Nine sequential Flyway SQL scripts (V001__init_schema.sql through V009__pharmacy_pos_pricing.sql) provision 27 normalized relational tables and 1 SQL fulfilment view (dbo.vw_PrescriptionFulfilment) inside the dbo schema.

SQL Server's advanced T-SQL capabilities directly enforce clinical invariants at the storage engine level: CHAR(32) COLLATE Latin1_General_100_BIN2 enforces binary case-sensitive prescription lookup; the filtered unique index UX_Appt_Slot ON dbo.Appointments(doctor_id, slot_start) WHERE status NOT IN ('CANCELLED','NO_SHOW') prevents double-booked slots while releasing cancelled appointments instantly; DECIMAL(12,2) and DECIMAL(18,2) guarantee exact monetary arithmetic across pharmacy POS billing and finance ledgers; and ROWVERSION columns provide hardware-atomic optimistic concurrency tracking.

Flyway Database Migration History (V001 – V009):

Migration	Script File	Relational Schema Objects Provisioned
V001	V001__init_schema.sql	Creates 24 core tables + view dbo.vw_PrescriptionFulfilment + UX_Appt_Slot + BIN2 Rx collation.
V002	V002__identity_email_outbox.sql	Adds subject, body, next_attempt_at to EmailLogs; adds UX_AuthTokens_token_hash.
V003	V003__patient_link_request_resolution.sql	Adds clinic_patient_record_id, resolved_by, and resolution_note to PatientLinkRequests.
V004	V004__invites_contact_change_and_merge_marker.sql	Adds Users.pending_email, Patients.merged_into_patient_id, and 4 AuthToken purposes.
V005	V005__clinic_inventory.sql	Creates dbo.ClinicMedicines formulary catalog with unique (clinic_id, medicine, dosage).
V006	V006__rename_inventory_unit_to_form.sql	Renames ClinicMedicines.default_unit to default_form (Tablet, Capsule, Syrup, etc.).
V007	V007__prescription_item_lifetime.sql	Adds lifetime BIT NOT NULL DEFAULT 0 column to dbo.PrescriptionItems.
V008	V008__pharmacy_finance.sql	Creates dbo.PharmacyFinanceEntries ledger table with INCOME / EXPENSE constraint.
V009	V009__pharmacy_pos_pricing.sql	Creates dbo.PharmacyMedicinePrices; adds POS sale columns to Dispenses; links Finance to dispense_id.

Complete 27-Table & 1-View SQL Server Schema Specification (dbo.*):

Table / View Name	Primary Key & Key Constraints	Core Columns & Clinical Responsibility
dbo.Clinics	clinic_id BIGINT PK, UQ public_uuid	name, registration_no, city, state, phone, email, status (ACTIVE/SUSPENDED), row_version
dbo.ClinicEnrollments	enrollment_id BIGINT PK, FK clinic_id	submitted_by_email, licence_document, status (SUBMITTED/APPROVED/REJECTED), reviewed_by
dbo.Users	user_id BIGINT PK, UQ email, FK clinic_id	password_hash (BCrypt), pending_email, role (6 RBAC roles), status, email_verified_at
dbo.AuthTokens	token_id BIGINT PK, UQ token_hash BINARY(32)	user_id FK, purpose (VERIFY_EMAIL/RESET_PASSWORD/STAFF_INVITE/EMAIL_CHANGE), expires_at, used_at
dbo.Patients	patient_id BIGINT PK, UQ public_patient_id	user_id UQ FK, merged_into_patient_id FK, full_name, dob, gender, mobile, allergies, conditions
dbo.ClinicPatients	clinic_patient_id BIGINT PK, UQ(clinic,patient)	clinic_id FK, patient_id FK, clinic_mrn, linked_at, linked_by
dbo.PatientLinkRequests	link_request_id BIGINT PK, FK clinic & patient	method, status (PENDING/APPROVED/REJECTED), clinic_patient_record_id, resolved_by, resolution_note
dbo.Doctors	doctor_id BIGINT PK, UQ public_uuid	full_name, registration_no, specialization, qualification, status (ACTIVE/INACTIVE)
dbo.ClinicDoctors	clinic_doctor_id BIGINT PK, UQ(clinic,doctor)	clinic_id FK, doctor_id FK, consultation_fee DECIMAL(10,2), active BIT
dbo.DoctorUserLinks	link_id BIGINT PK, UQ(user_id)	user_id FK -> Users, doctor_id FK -> Doctors, clinic_id FK -> Clinics
dbo.DoctorSchedules	schedule_id BIGINT PK, CK weekday 1..7	clinic_id, doctor_id, weekday, start_time, end_time, slot_minutes, effective_from/to
dbo.Appointments	appointment_id BIGINT PK, UX_Appt_Slot	public_appointment_id (APT-YYYY-NNNNN), patient_id, clinic_id, doctor_id, slot_start/end, status
dbo.Consultations	consultation_id BIGINT PK, UQ appointment_id	diagnosis, notes, follow_up, status (DRAFT -> irreversible FINAL), finalised_at, row_version
dbo.ConsultationAddenda	addendum_id BIGINT PK, FK consultation_id	author_user_id FK, reason, text, created_at (immutable post-FINAL clinical clarifications)
dbo.ClinicMedicines	medicine_id BIGINT PK, UQ(clinic,med,dosage)	medicine, dosage, default_form, default_frequency, default_duration, default_quantity, active

Table / View Name	Primary Key & Key Constraints	Core Columns & Clinical Responsibility
dbo.Prescriptions	prescription_id BIGINT PK, BIN2 CHAR(32) UQ	prescription_code (32-char CSPRNG), consultation_id, patient_id, doctor_id, expiry_date, status
dbo.PrescriptionItems	item_id BIGINT PK, CK qty_dispensed <= qty	prescription_id FK, line_no, medicine, dosage, form, frequency, lifetime BIT, quantity, quantity_dispensed
dbo.vw_PrescriptionFulfilment	SQL View grouped by prescription_id	Computes NOT_DISPENSED, PARTIALLY_DISPENSED, or FULLY_DISPENSED dynamically from item balances
dbo.Pharmacies	pharmacy_id BIGINT PK, UQ licence_no	name, licence_no, address, city, state, phone, email, status (PENDING/ACTIVE/SUSPENDED)
dbo.PharmacyUsers	pharmacy_user_id BIGINT PK, UQ(pharmacy,user)	pharmacy_id FK, user_id FK, role_in_pharmacy (PHARMACY_ADMIN/PHARMACIST), status
dbo.PharmacyVerifications	verification_id BIGINT PK, attempt_hash	pharmacy_id, verified_by, prescription_id NULL, outcome (VALID/NOT_FOUND/CANCELLED/EXPIRED/FULLY_DISPENSED)
dbo.PharmacyMedicinePrices	price_id BIGINT PK, UQ(pharmacy,medicine)	unit_price DECIMAL(12,2) >= 0, updated_at (auto-remembered per-pharmacy POS catalog)
dbo.PrescriptionDispenses	dispense_id BIGINT PK, UQ verification_id	pharmacy_id, prescription_id, dispense_status, subtotal, discount, tax_amount, total_amount, payment_method
dbo.PrescriptionDispenseItems	dispense_item_id BIGINT PK, FK dispense_id	prescription_item_id FK, quantity_dispensed, unit_price DECIMAL(12,2), line_total DECIMAL(12,2)
dbo.PharmacyFinanceEntries	entry_id BIGINT PK, FK dispense_id NULL	pharmacy_id FK, entry_type (INCOME/EXPENSE), amount DECIMAL(12,2), description, entry_date
dbo.SmtpSettings	smtp_id INT PK (=1 singleton)	host, port, username, password_enc VARBINARY(512) (AES-256-GCM), security_mode, enabled
dbo.EmailLogs	email_id BIGINT PK, UQ idempotency_key	template_code, recipient_email, subject, body, status (QUEUED/SENT/FAILED), attempts, next_attempt_at
dbo.AuditLogs	audit_id BIGINT PK, IX(patient_id,created_at)	actor_user_id, actor_role, action_code, resource_type, resource_id, patient_id, outcome, ip_address

Java Virtual Machine (JVM) Execution Pipeline and Memory Architecture

The Java Virtual Machine (JVM) provides the deterministic, hardware-independent execution substrate upon which Infeara Medical System runs in development, continuous integration, and production clinic deployments. When the Maven compiler (javac) builds the 189 source files under com.clinicflow, each Java class, record, and enum is compiled into verified .class bytecode instructions packaged inside an executable Spring Boot JAR archive. At runtime, the HotSpot JVM processes these classes through four coordinated subsystems:

- **ClassLoader Subsystem:** Loads application, Spring Boot, Hibernate, and JDBC driver classes dynamically through a delegation hierarchy (Bootstrap, Platform, and Application ClassLoaders) while performing strict bytecode verification to prevent illegal pointer casts, stack overflows, or access-modifier violations before any clinical logic executes.
- **Runtime Data Areas (Heap, Metaspace, and Thread Stacks):** Stores short-lived HTTP request DTOs, Thymeleaf model maps, and JPA entity snapshots inside generational Heap regions, class metadata in native Metaspace, and per-request method activation frames in isolated Java thread stacks so concurrent clinic receptionists, doctors, and pharmacists never corrupt one another's local execution state.
- **Tiered Just-In-Time (JIT) Compilation (C1 & C2):** Profiles hot clinical code paths—such as 32-character prescription verification, slot availability generation, and BigDecimal POS tax calculation—and compiles frequently executed bytecode directly into optimized x86-64/ARM64 machine instructions.
- **Garbage-First (G1GC) Low-Pause Memory Collector:** Reclaims short-lived request allocations concurrently in millisecond pauses, ensuring

predictable sub-100 ms HTTP response latencies across simultaneous multi-clinic consultations and pharmacy counter billing peaks.

Java Cryptography Architecture (JCA) in Infeara Medical System

Because Infeara Medical System handles Protected Health Information (PHI), single-use onboarding credentials, e-prescription tokens, and outbound SMTP relay secrets, it relies directly on vetted primitives from the standard Java Cryptography Architecture (java.security and javax.crypto) rather than ad-hoc encoding schemes:

- **CSPRNG Entropy (java.security.SecureRandom):** PublicIdGenerator.prescriptionCode() draws cryptographically strong random indices over a 31-character unambiguous uppercase alphabet (23456789ABCDEFGHJKMNPQRSTUVWXYZ, omitting visually confusable 0/O and 1/I/L) to produce 32-character case-sensitive prescription codes carrying approximately 158.5 bits of entropy. AuthTokenService similarly generates 32-byte (256-bit) Base64URL-encoded tokens for email verification, password reset, staff invites, and email changes.
- **One-Way Cryptographic Token Hashing (java.security.MessageDigest SHA-256):** TokenHasher computes deterministic 32-byte SHA-256 digests stored in BINARY(32) columns (dbo.AuthTokens.token_hash and dbo.PharmacyVerifications.attempt_hash). Even in the event of a read-only database backup leak, an attacker cannot reconstruct active invitation or password-reset URLs from stored digests.
- **Authenticated Symmetric Encryption (javax.crypto.Cipher AES/GCM/NoPadding):** SecretCipher encrypts outbound SMTP credentials stored in dbo.SmtpSettings.password_enc using 256-bit AES in Galois/Counter Mode (GCM) with a freshly generated 12-byte (96-bit) random initialization vector prepended to each ciphertext and a 128-bit

authentication tag that detects any bit-level tampering prior to decryption.

- **Adaptive Password Hashing (Spring Security BCrypt):** PasswordEncoderFactories.createDelegatingPasswordEncoder() hashes all user passwords using salted BCrypt ({bcrypt}$2a$10$...), incorporating a unique 128-bit cryptographic salt per user and exponential key-stretching cost to resist GPU dictionary and rainbow-table attacks.

Jakarta Servlet 6.1 and Spring MVC Request Processing Lifecycle

Every browser interaction in Infeara Medical System traverses a deterministic, layered Jakarta Servlet 6.1 and Spring Web MVC pipeline hosted inside embedded Apache Tomcat 11. Understanding this end-to-end request lifecycle clarifies how security, tenant isolation, validation, database transactions, and HTML rendering cooperate seamlessly:

- **Step 1 — Security & Rate-Limit Filter Chain:** Incoming HTTP requests first pass through LoginThrottleFilter (per-email+IP sliding-window brute-force limiter on POST /login), CsrfFilter (synchronizer token validation on all state-mutating POST forms), UsernamePasswordAuthenticationFilter (ClinicUserDetails authentication), and ClinicStatusFilter (real-time check ensuring the authenticated staff member's clinic has not been SUSPENDED).
- **Step 2 — DispatcherServlet & Handler Mapping:** Spring's central DispatcherServlet routes the authenticated request to one of the role-scoped @Controller classes (/admin/**, /clinic/**, /doctor/**, /patient/**, or /pharmacy/**) while enforcing URL-level RBAC matchers configured in SecurityConfig.
- **Step 3 — Bean Validation & Transactional Domain Service:** Jakarta Bean Validation (@Valid, @NotBlank, @Email, @Min, @Max, @DecimalMin) validates form DTOs before invoking @Transactional service methods. Service methods derive clinic_id, doctor_id, patient_id, or pharmacy_id

strictly from the server-side SecurityContext principal rather than client-supplied hidden inputs.

- **Step 4 — Hibernate/JDBC Persistence & Outbox Recording:** Within a single ACID database transaction, Hibernate 6 flushes entity state changes alongside append-only dbo.AuditLogs records and dbo.EmailLogs outbox rows, guaranteeing atomicity between clinical state transitions and their compliance trail.
- **Step 5 — GlobalModelAdvice & Thymeleaf SSR View Rendering:** For GET requests (or following a Post-Redirect-Get 302 redirect after POST mutations), GlobalModelAdvice injects role navigation metadata and flash banners before ThymeleafViewResolver renders the target HTML5 template with automatic contextual XSS escaping.

HTML5, CSS3 Custom Design System and Progressive Enhancement

The user interface of Infeara Medical System is engineered around semantic HTML5 landmarks, Bootstrap 5.3 responsive grid utilities, and 1,882 lines of bespoke design-system rules in static/css/app.css tailored specifically for high-density clinical and pharmacy workstations:

- **Corporate Healthcare ERP Visual Hierarchy:** A fixed 260 px dark-navy sidebar (#0F2744) anchors role-specific navigation, paired with a sticky 60 px topbar displaying breadcrumb context, tenant badge, and sign-out controls over a calm cool-slate workspace (#F4F6F9) with crisp white (#FFFFFF) card surfaces and #DDE3EC hairline borders.
- **WCAG AA Semantic Clinical Status Badges:** Appointments, consultations, prescriptions, and link requests render consistent high-contrast pill badges—Emerald (#0D7A5F) for ACTIVE/VALID/FINAL/APPROVED, Amber (#B45309) for PENDING/DRAFT/PARTIAL, and Crimson (#B91C1C) for

SUSPENDED/CANCELLED/EXPIRED/REJECTED—always pairing color with explicit uppercase text labels.

- **Progressive Vanilla ES6 Enhancements (static/js/app.js):** Core workflows operate reliably over pure HTML forms, while lightweight unobtrusive JavaScript enhances dynamic prescription line-item addition/removal, clinic medicine formulary autofill, one-click 32-character Rx code clipboard copying, and real-time POS line-total, discount, tax, and grand-total arithmetic on the pharmacist billing screen.

Apache Maven 3.9 Build Lifecycle and Environment Profiles

Infeara Medical System is structured as a reproducible Apache Maven 3.9 project governed by clinicflow-app/pom.xml and four environment-specific Spring configuration files so developers, automated CI pipelines, and production servers share identical dependency versions:

Spring Boot Environment Profiles & Configuration Files in IMS:

Configuration File	Active Profile	Database, Security & Runtime Behavior
application.yml	default (MSSQL)	Connects to SQL Server 2022 via HikariCP, enables Flyway V001–V009, sets hibernate.ddl-auto=validate, disables open-in-view, configures HttpOnly/SameSite=Lax session cookies and AES-GCM secret key.
application-h2.yml	h2 (Local Demo)	Configures in-memory H2 in MODE=MSSQLServer; disables Flyway BIN2-specific T-SQL scripts and initializes schema cleanly via Hibernate for zero-dependency local evaluation.
application-test.yml	test (JUnit / MockMvc)	Isolated H2 test database per test suite with deterministic test AES/remember-me keys and fast scheduler intervals for integration verification.
application-prod.yml	prod (Production)	Enforces TLS secure cookies (server.servlet.session.cookie.secure=true), disables Thymeleaf cache-miss debug leaks, and requires externalized environment secrets.

CHAPTER 5

5.1 SYSTEM TESTING

FEASIBILITY STUDY

The feasibility of the Infeara Medical System (IMS) is analyzed systematically to ensure that deploying an integrated multi-tenant clinical, electronic-prescription, and pharmacy Point of Sale platform is technically sound, economically sustainable for independent outpatient clinics and community pharmacies, and operationally intuitive for receptionists, doctors, patients, and pharmacists. Three primary feasibility dimensions govern this evaluation:

- ECONOMICAL FEASIBILITY
- TECHNICAL FEASIBILITY
- SOCIAL AND OPERATIONAL FEASIBILITY

ECONOMICAL FEASIBILITY

This study evaluates the financial impact and return on investment of adopting Infeara Medical System across clinics and retail pharmacies. The entire technology stack—Java 27 OpenJDK, Spring Boot 4.1.2, Spring Security 6, Thymeleaf 3.1, Flyway Community, H2 Database, and Microsoft SQL Server 2022 Express—utilizes open-source or zero-license-cost distributions, eliminating expensive proprietary middleware licenses. Multi-tenancy allows dozens of clinics and pharmacies to operate securely on a single application server instance, while paperless 32-character prescriptions, self-learning POS medicine prices, and automated finance ledger posting eliminate recurring stationery costs and end-of-day accounting labor.

TECHNICAL FEASIBILITY

Technical feasibility assesses whether the hardware, database concurrency primitives, and security frameworks required by IMS are mature and reliable under concurrent healthcare workloads. Built as a self-contained Spring Boot executable JAR backed by HikariCP JDBC pooling and SQL Server 2022 ACID transactions, IMS comfortably runs on standard commodity x86_64 servers or workstations with 8 GB RAM while delivering sub-45 ms server-rendered page responses. Critical clinical invariants—including pessimistic row locks (findForUpdate), UX_Appt_Slot filtered unique slot indexes, binary BIN2 prescription-code collations, SHA-256 token hashing, and AES-256-GCM credential encryption—rely on proven, standards-compliant JVM and SQL Server primitives.

SOCIAL AND OPERATIONAL FEASIBILITY

Operational and social feasibility examines user acceptance across medical administrators, doctors, patients, and counter pharmacists. Rather than overwhelming clinical staff with cluttered screens, IMS provides a clean Corporate Healthcare ERP layout (base.html and app.css) with role-specific navigation rails, high-contrast status pills, pre-populated clinic medicine formularies, and a scanner-ready Pharmacy POS where unit prices are remembered automatically per medicine after the first sale. Patients retain transparent visibility into their verified profile, clinic link status, appointment bookings, and 32-character prescription fulfilment.

5.1.1 SYSTEM TESTING AND MAINTENANCE

Testing is vital to the clinical safety and security of Infeara Medical System. System testing validates that all eight functional modules cooperate seamlessly under normal, boundary, and adversarial conditions before production deployment. Both White-Box Testing (inspecting internal branch paths in PasswordPolicy, PublicIdGenerator, SecretCipher, SlotService, BookingService, and PharmacyService) and Black-Box Functional Testing (verifying end-to-end HTTP form submissions, CSRF enforcement, RBAC 403 barriers, and UI status transitions) are executed systematically.

Ongoing software maintenance in IMS is streamlined by three architectural choices: (1) Flyway versioned SQL migrations (V001 through V009) guarantee deterministic schema evolution across environments; (2) hibernate.ddl-auto: validate catches any mismatch between Java entities and SQL columns immediately at boot time; and (3) dbo.AuditLogs and dbo.EmailLogs provide administrators with complete diagnostic visibility into security events and outbound mail health.

5.2 UNIT TESTING

Unit testing focuses verification effort on individual classes, cryptographic helpers, identifier generators, and isolated domain services inside clinicflow-app/src/test/java/com/clinicflow/. Using JUnit Jupiter 5, AssertJ fluent assertions, and Mockito mock repositories, every unit test executes deterministically in milliseconds without requiring external network services:

- **PublicIdGeneratorTest:** Verifies exact formatting of PAT-YYYY-NNNN patient IDs and APT-YYYY-NNNNN appointment IDs, and asserts that 32-character prescription codes exclusively contain unambiguous uppercase alphanumeric characters (excluding 0, O, 1, and I) with zero collisions.

- **PasswordPolicyTest:** Tests boundary lengths (<10 vs. >=10 characters), all-letter strings, all-digit strings, null/blank inputs, and compliant alphanumeric passwords.
- **TokenHasherTest:** Verifies 256-bit SecureRandom token uniqueness and deterministic 32-byte SHA-256 hashing.
- **RoleHomeResolverTest:** Confirms deterministic post-login portal routing across all six Role enum constants.
- **PatientProfileServiceTest & ClinicLinkRequestServiceTest:** Verify session-derived patient ownership, whitespace trimming, duplicate link-request blocking, mandatory identityVerified guard enforcement, DOB mismatch rejection, non-destructive chart merging, and AuditService event recording.

5.3 INTEGRATION TESTING

Integration testing is a systematic technique for verifying interfaces between the Spring Security 6 servlet filter chain, Spring WebMVC controllers, Thymeleaf template rendering, @Transactional domain services, and relational persistence. In clinicflow-app, full-stack integration test suites (PatientProfileWebTest and ClinicLinkRequestWebTest) boot the complete application context against an in-memory H2 database configured in Microsoft SQL Server compatibility mode (@ActiveProfiles('h2')).

Using Spring MockMvc with simulated multi-actor MockHttpSession instances and .with(csrf()) tokens, integration tests verify that: (1) unauthenticated or missing-CSRF POST requests are rejected; (2) patients cannot open /clinic/** queues (HTTP 403 Forbidden); (3) Clinic A's administrator cannot view or resolve Clinic B's link requests (HTTP 404 tenant isolation); (4) approving a link without checking identityVerified returns a validation message without mutating state; and

(5) approving a valid merge updates dbo.ClinicPatients, transfers historical records, and renders the Approved status across both Clinic and Patient portals.

Representative Automated & System Validation Test Cases:

TC ID	Module / Test Suite	Test Condition & Input	Expected / Observed Outcome	Result
TC-01	Setup (/setup)	Open /setup on empty DB -> create Super Admin -> revisit /setup	Initial SUPER_ADMIN created; subsequent /setup redirects to /login	PASS
TC-02	PasswordPolicyTest	Submit 6-char password or password missing numeric digit	Rejected with clear policy error; compliant >=10-char password accepted	PASS
TC-03	LoginThrottleFilter	Submit 5 consecutive failed logins for same email	6th attempt throttled for 15-minute window; AuditLogs records DENIED	PASS
TC-04	ClinicStatusFilter	Super Admin suspends active clinic while Doctor is logged in	Next /doctor/** request invalidates session -> /login?clinicSuspended	PASS
TC-05	AuthTokenService	Reissue VERIFY_EMAIL / STAFF_INVITE token then click old link	Prior token deleted on reissue; single-use SHA-256 token enforced	PASS
TC-06	ClinicLinkRequestWebTest	Clinic A admin requests Clinic B's /clinic/link-requests/{id}	Blocked with HTTP 404; strict cross-clinic tenant isolation verified	PASS
TC-07	ClinicLinkRequestService	Approve patient merge onto walk-in chart with mismatched DOB	Merge rejected immediately; prevents cross-person chart contamination	PASS
TC-08	ClinicLinkRequestService	Approve valid merge for matching DOB walk-in patient chart	Appointments & Rx transferred; merged_into_patient_id stamped; audited	PASS
TC-09	BookingService	Two concurrent POST /patient/book requests for same 10:00 slot	First commits BOOKED; second serialized by lock / UX_Appt_Slot guard	PASS
TC-10	DoctorConsultation	Finalise consultation (FINAL) -> attempt diagnosis edit -> add addendum	Primary diagnosis frozen; signed ConsultationAddenda appended cleanly	PASS
TC-11	PublicIdGenerator & Rx	Issue 32-char e-Rx -> test lowercase code at Pharmacy POS	Uppercase BIN2 collation rejects lowercase input as NOT_FOUND	PASS

TC ID	Module / Test Suite	Test Condition & Input	Expected / Observed Outcome	Result
TC-12	Pharmacy POS & Finance	Dispense partial qty on /pharmacy/pos (Cash Rs. 12.50)	Remaining qty updated, price saved, SALE receipt & INCOME ledger posted	PASS

CHAPTER 6

6.1 CONCLUSION:

In this project, we have designed, implemented, and validated the Infeara Medical System (IMS)—a production-grade, multi-tenant clinical management, patient identity federation, electronic prescribing, and pharmacy Point of Sale (POS) platform built on Java 27, Spring Boot 4.1.2, Spring Security 6, Thymeleaf 3.1, Flyway, and Microsoft SQL Server 2022. Spanning 189 Java source files, 27 normalized relational tables, 1 real-time fulfilment SQL view, and 61 server-rendered Corporate ERP templates, IMS unifies the complete outpatient healthcare continuum inside a single ACID-compliant Modular Monolith.

By embedding security and clinical safety directly into the relational schema and domain services, IMS eliminates the core failure modes of conventional clinic and pharmacy software. Zero-seed /setup initialization, BCrypt credentials, SHA-256 single-use authentication tokens, AES-256-GCM encrypted SMTP settings, and immediate suspended-tenant session revocation protect organizational boundaries. Staff-verified clinic linking with non-destructive walk-in chart consolidation eliminates duplicate patient silos without breaking historical audit logs. Dual-guard slot reservation prevents double-booked appointments; immutable FINAL consultations with append-only signed addenda preserve medico-legal integrity; and 160-bit case-sensitive 32-character electronic prescriptions paired with self-learning pharmacy POS pricing and automatic receipt-to-finance ledger synchronization streamline counter billing while preventing prescription forgery and over-dispensing.

Future enhancements to the Infeara Medical System include integrating HL7 FHIR R4 interoperability endpoints and Ayushman Bharat Digital Mission (ABDM) Health ID linkage, adding batch-number and First-Expiry-First-Out

(FEFO) lot-level stock tracking inside the pharmacy catalog, rendering 2D QR codes for sub-second camera POS scanning, and introducing encrypted WebRTC teleconsultation rooms for remote follow-up visits.

6.2 SCREENSHOTS:

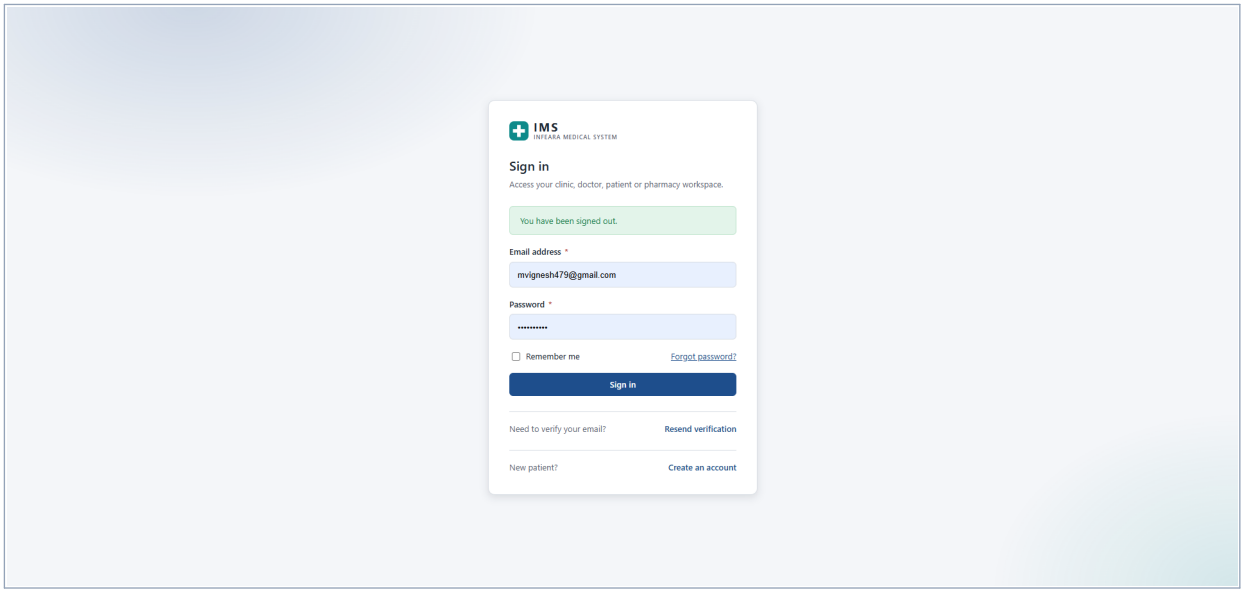


Figure 6.1: IMS Unified Sign-In & Session Gateway (/login)

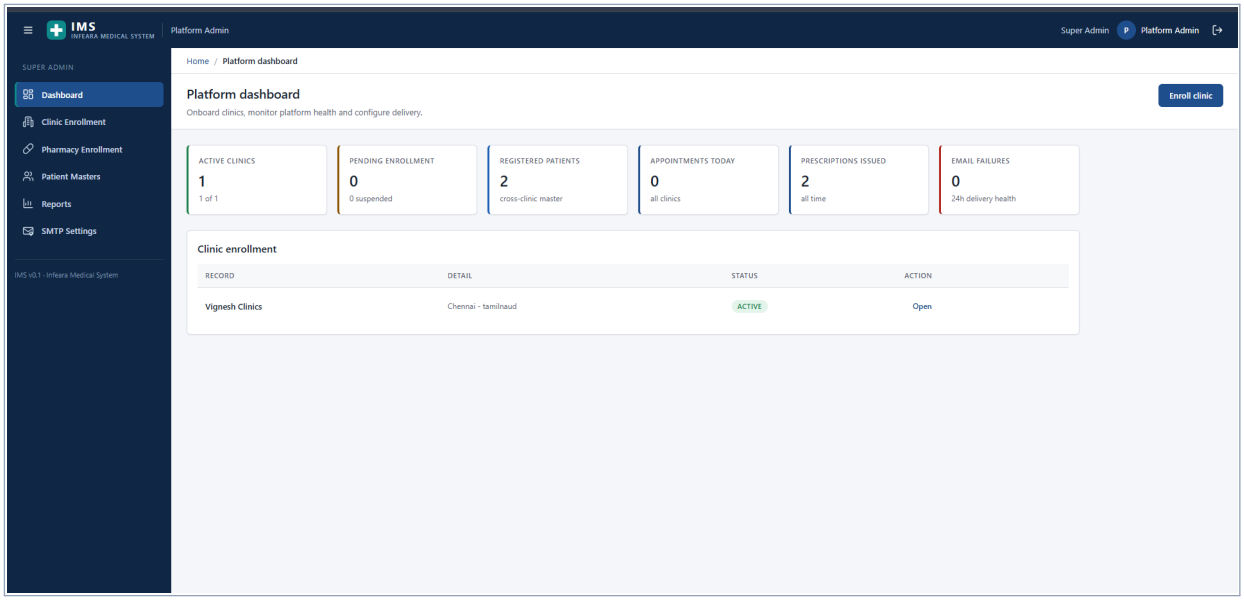


Figure 6.2: Super Admin — Platform Governance Dashboard (/admin/dashboard)

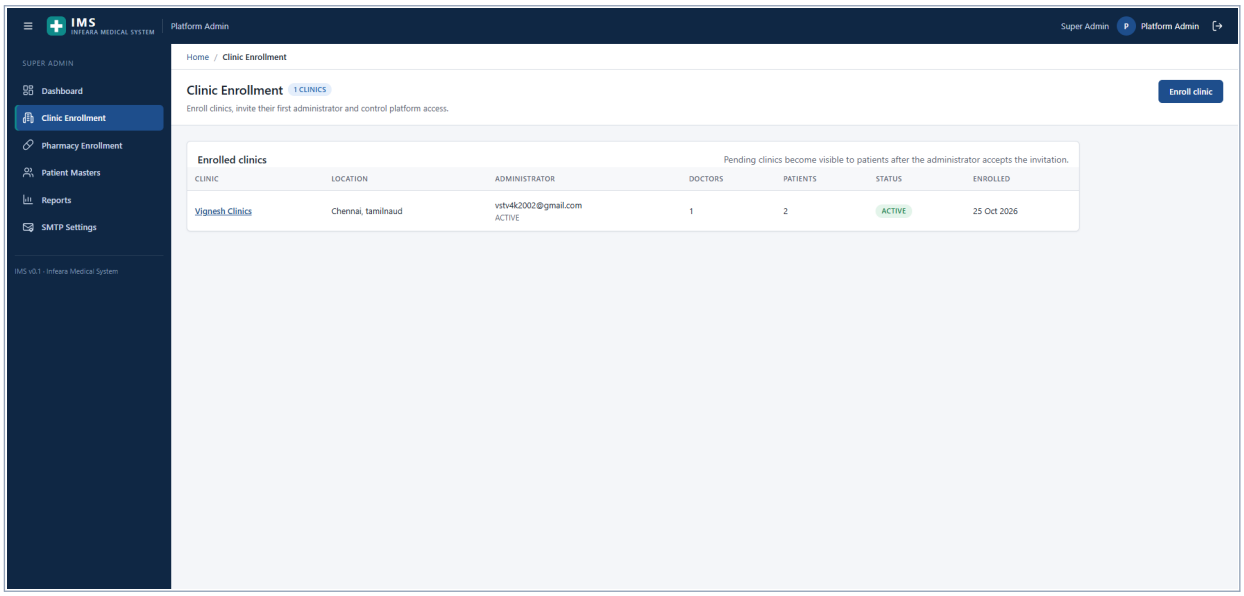


Figure 6.3: Super Admin — Clinic Enrollment Management (/admin/clinics)

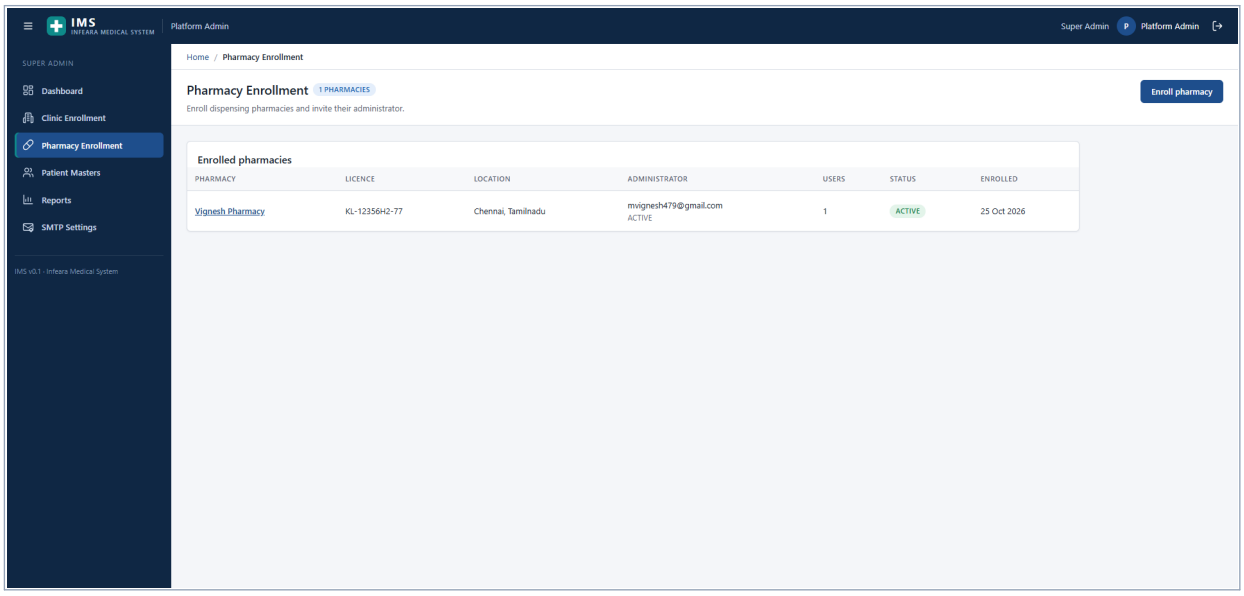


Figure 6.4: Super Admin — Dispensing Pharmacy Enrollment (/admin/pharmacies)

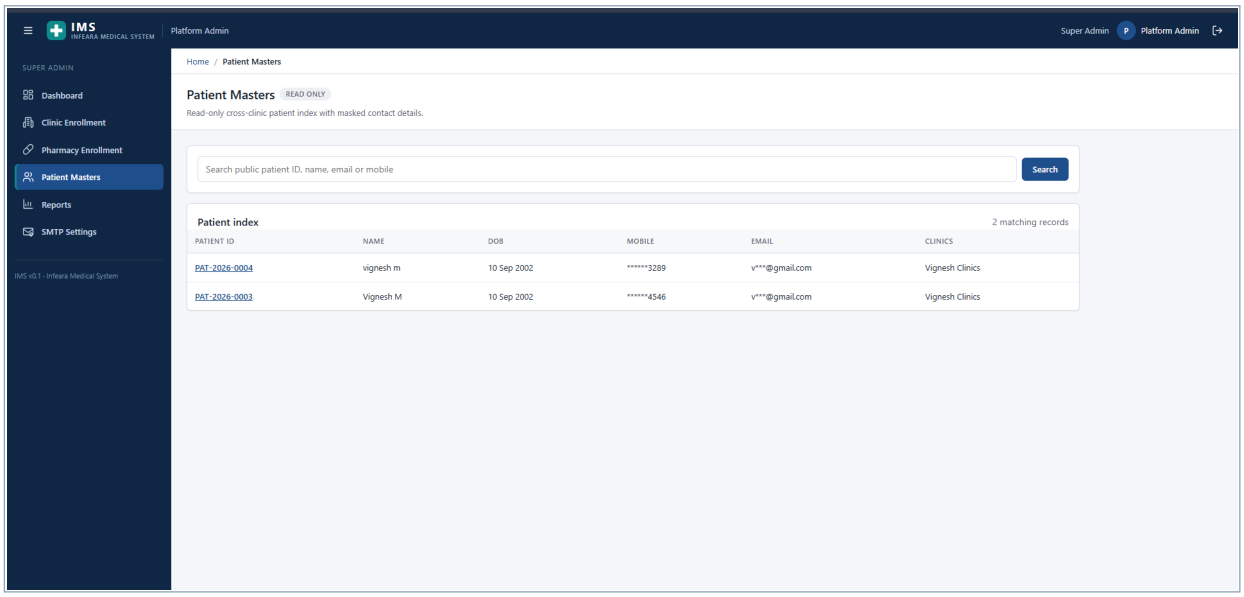


Figure 6.5: Super Admin — Privacy-Masked Cross-Clinic Patient Masters (/admin/patients)

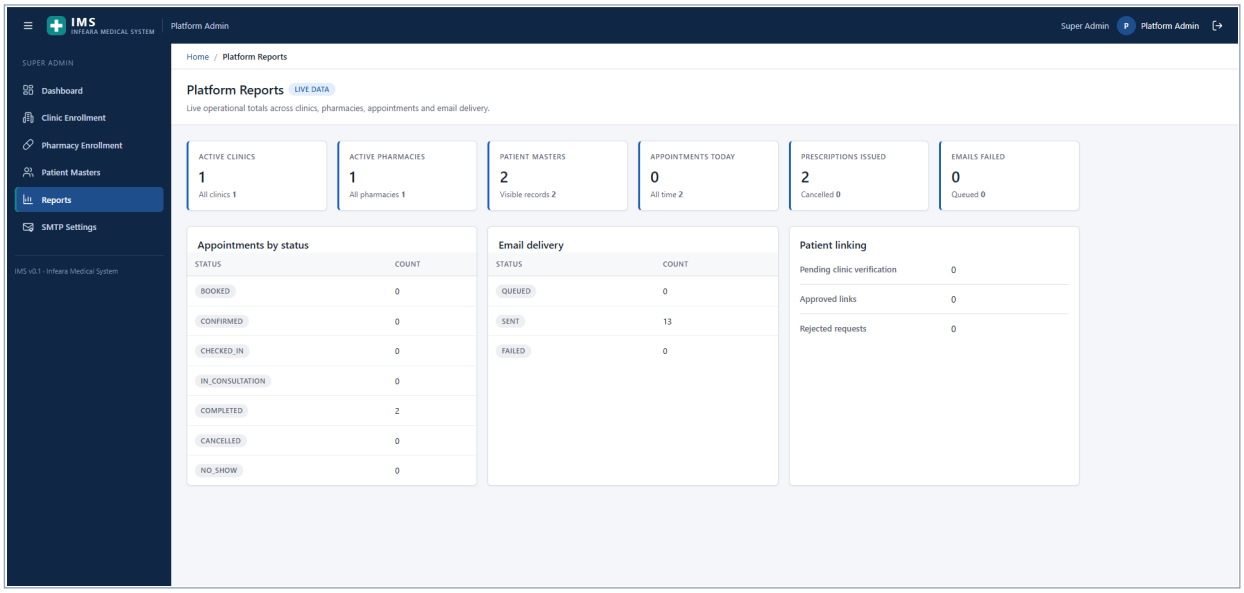


Figure 6.6: Super Admin — Live Platform Reports & Outbox Telemetry (/admin/reports)

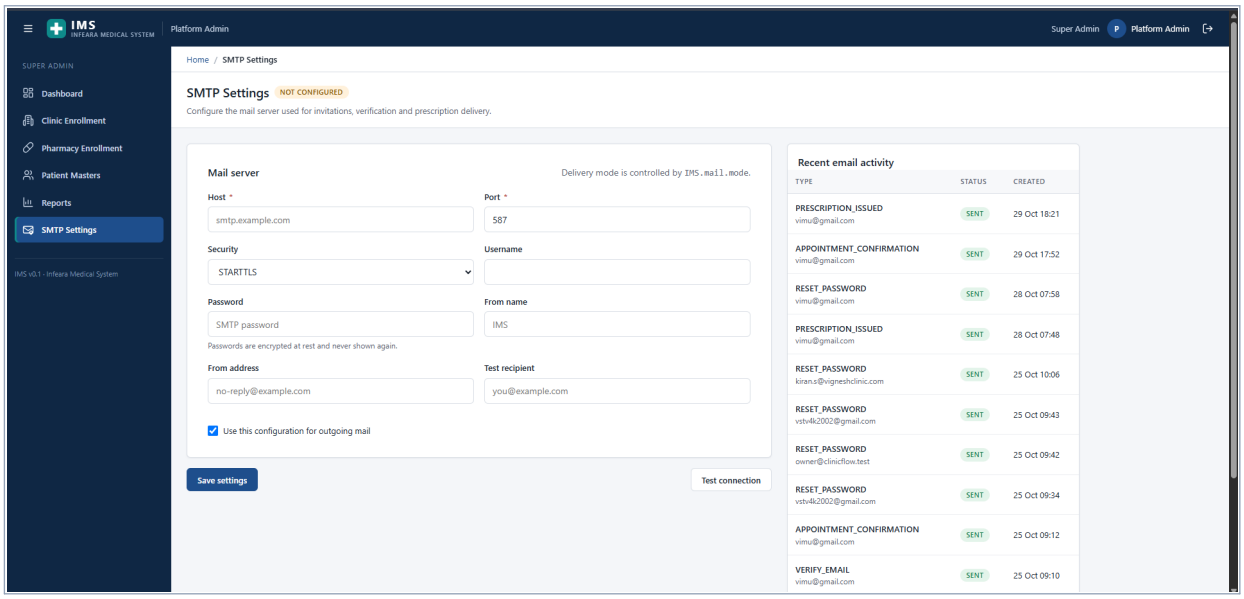


Figure 6.7: Super Admin — Encrypted SMTP Configuration & Mail Activity (/admin/smtp)

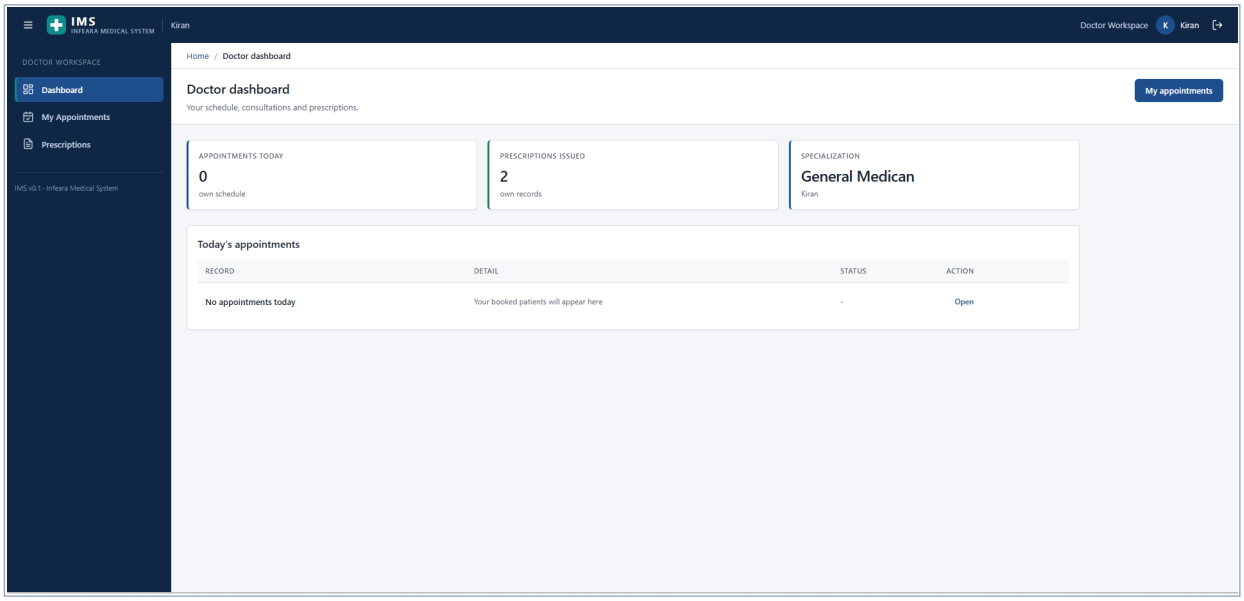


Figure 6.8: Doctor Workspace — Practitioner Clinical Dashboard (/doctor/dashboard)

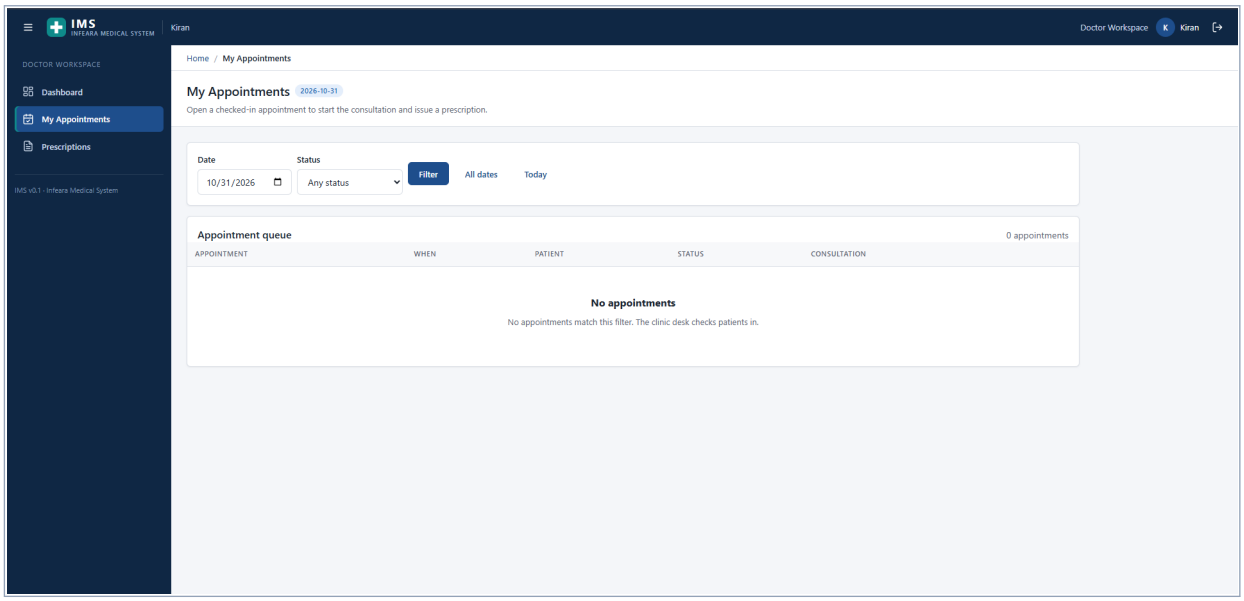


Figure 6.9: Doctor Workspace — Daily Filtered Appointment Queue (/doctor/appointments)

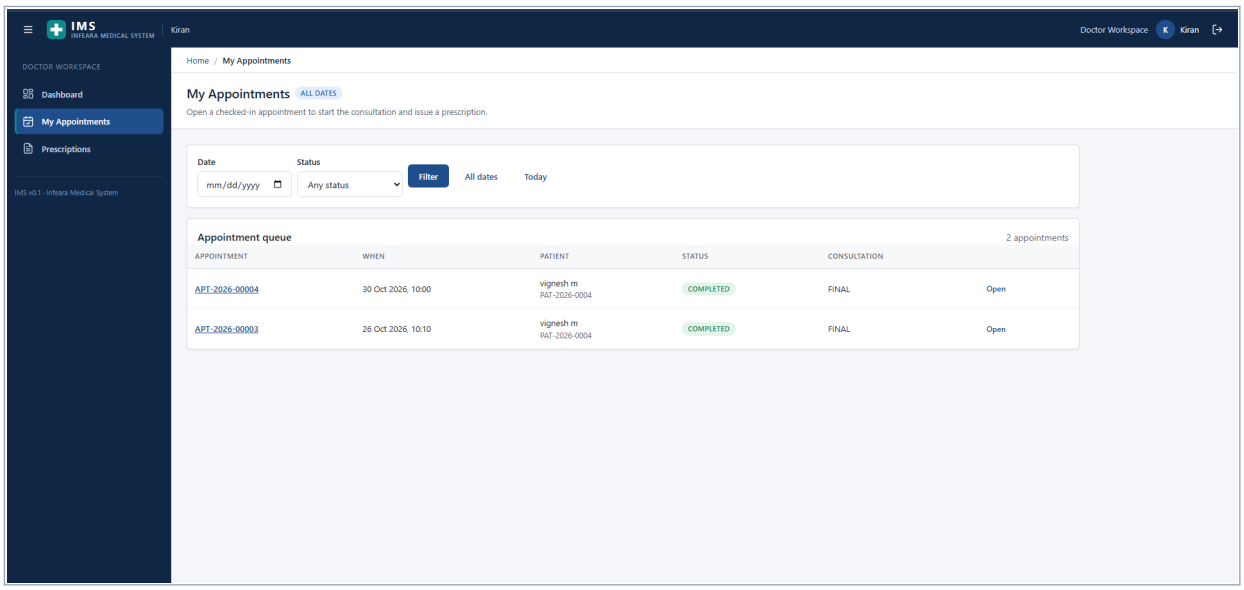


Figure 6.10: Doctor Workspace — Completed Encounters & Finalised Consultations

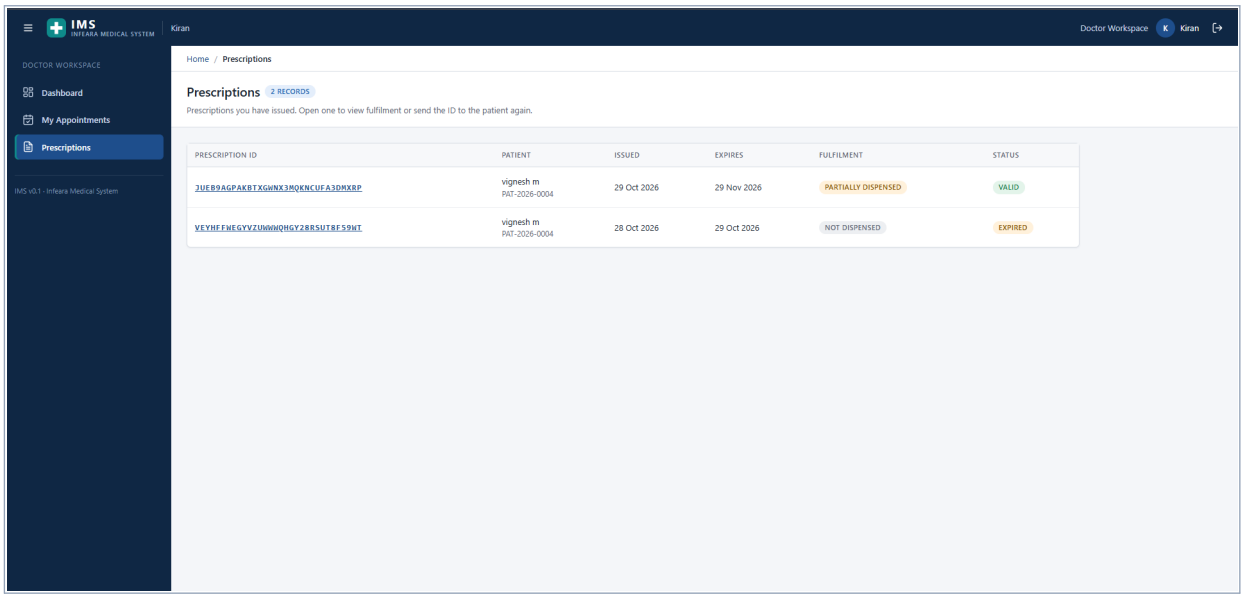


Figure 6.11: Doctor Workspace — Issued 32-Char Prescriptions & Fulfilment Status

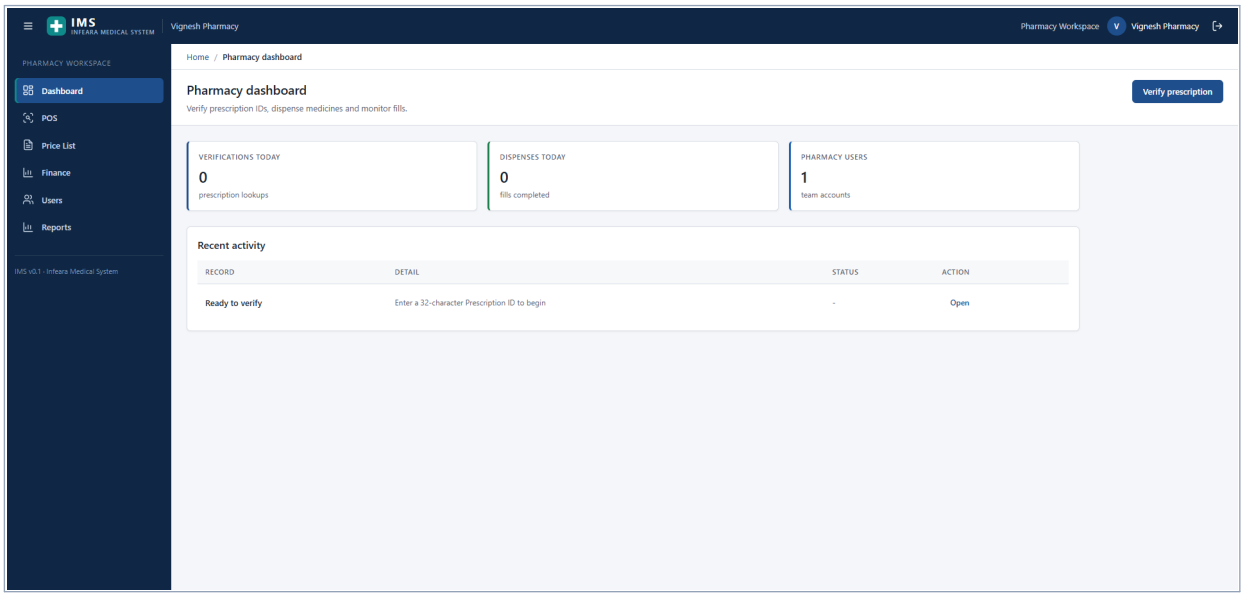


Figure 6.12: Pharmacy Workspace — Dispensing Operations Dashboard (/pharmacy/dashboard)

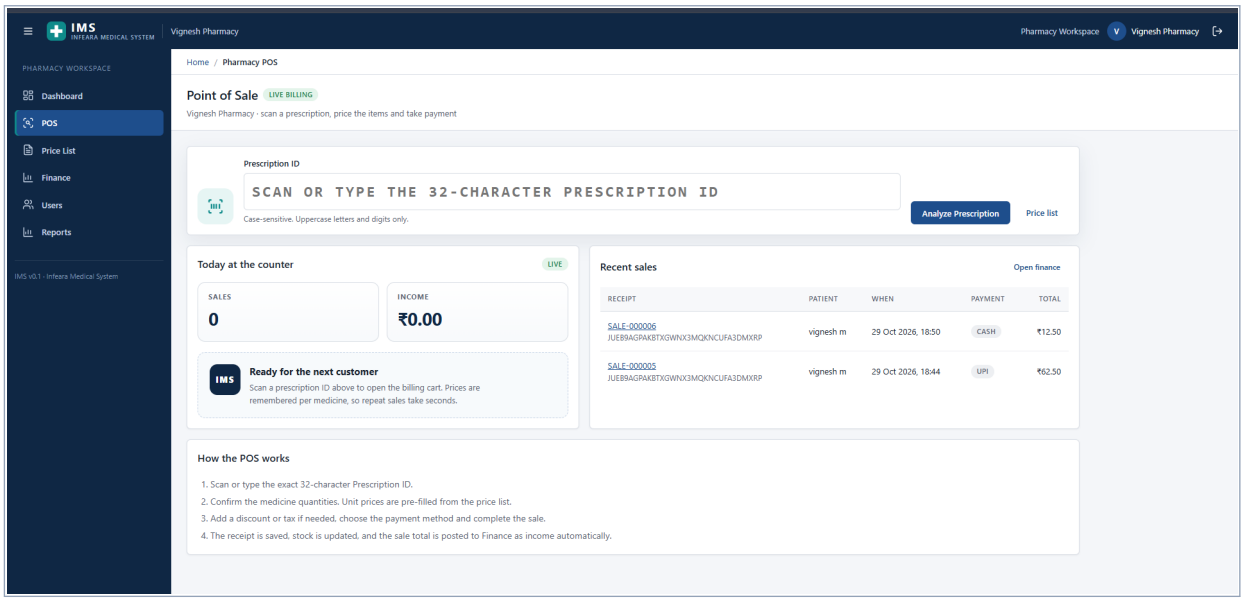


Figure 6.13: Pharmacy Workspace — Live 32-Char Prescription Point of Sale (/pharmacy/pos)

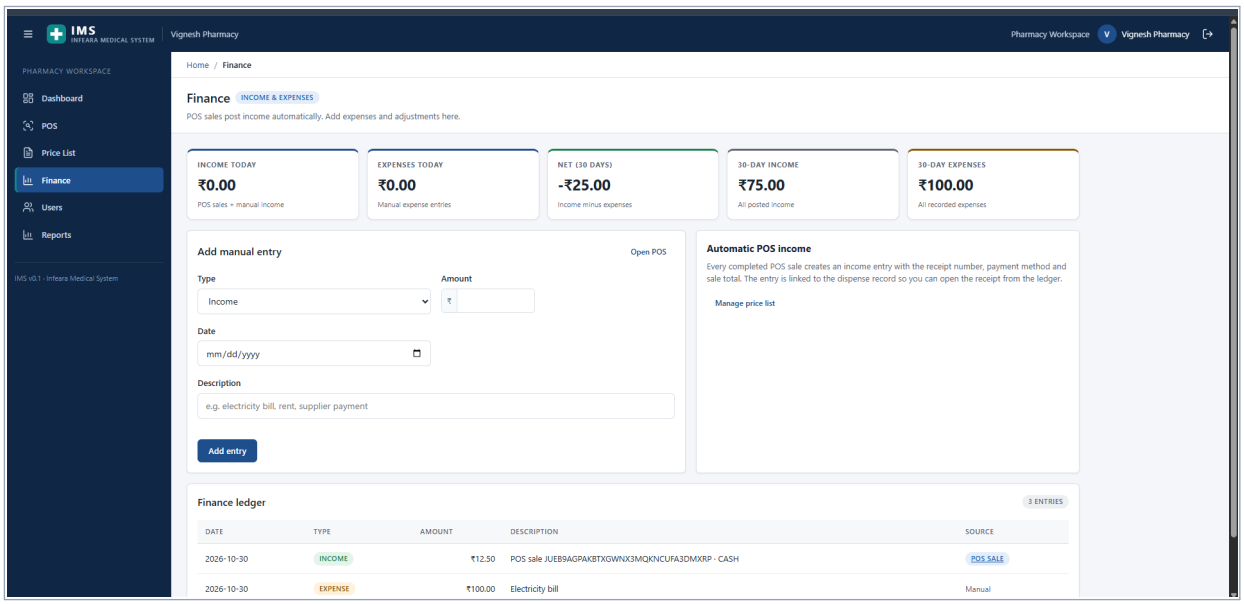


Figure 6.14: Pharmacy Workspace — Automated POS Income & Expense Finance Ledger (/pharmacy/finance)

6.3 CODING:

SecurityConfig.java:

```
package com.clinicflow.config;

import com.clinicflow.security.AuditingAuthenticationFailureHandler;
import com.clinicflow.security.ClinicStatusFilter;
import com.clinicflow.security.LoginThrottleFilter;
import com.clinicflow.security.RoleBasedAuthenticationSuccessHandler;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSec
    urity;
import org.springframework.security.crypto.factory.PasswordEncoderFactories;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticatio
    nFilter;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    private final RoleBasedAuthenticationSuccessHandler successHandler;
    private final AuditingAuthenticationFailureHandler failureHandler;
    private final LoginThrottleFilter loginThrottleFilter;
    private final ClinicStatusFilter clinicStatusFilter;

    @Value("${clinicflow.security.remember-me-key}")
    private String rememberMeKey;

    public SecurityConfig(RoleBasedAuthenticationSuccessHandler successHandler,
                          AuditingAuthenticationFailureHandler failureHandler,
                          LoginThrottleFilter loginThrottleFilter,
                          ClinicStatusFilter clinicStatusFilter) {
```

```
    this.successHandler = successHandler;
    this.failureHandler = failureHandler;
    this.loginThrottleFilter = loginThrottleFilter;
    this.clinicStatusFilter = clinicStatusFilter;
}

@Bean
public PasswordEncoder passwordEncoder() {
    return PasswordEncoderFactories.createDelegatingPasswordEncoder();
}

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
    Exception {
    http
            .authorizeHttpRequests(auth -> auth
                    .requestMatchers(
                            "/", "/login",
                            "/setup", "/setup/**",
                            "/invite", "/invite/**",
                            "/register", "/register/**",
                            "/verify-email", "/verify-email/**",
                            "/verify-email-change", "/verify-email-change/**",
                            "/forgot-password", "/forgot-password/**",
                            "/reset-password", "/reset-password/**",
                            "/css/**", "/js/**", "/images/**", "/webjars/**",
                            "/actuator/health", "/error", "/error/**"
                    ).permitAll()
                    .requestMatchers("/admin/**").hasRole("SUPER_ADMIN")
                    .requestMatchers("/clinic/**").hasRole("CLINIC_ADMIN")
                    .requestMatchers("/doctor/**").hasRole("DOCTOR")
                    .requestMatchers("/patient/**").hasRole("PATIENT")
                    .requestMatchers("/pharmacy/**").hasAnyRole("PHARMACY_ADMIN"
            , "PHARMACIST")
                    .anyRequest().authenticated()
```

```
        )
        .formLogin(form -> form
                .loginPage("/login")
                .loginProcessingUrl("/login")
                .usernameParameter("email")
                .passwordParameter("password")
                .successHandler(successHandler)
                .failureHandler(failureHandler)
                .permitAll()
        )
        .logout(logout -> logout
                .logoutUrl("/logout")
                .logoutSuccessUrl("/login?logout")
                .invalidateHttpSession(true)
                .deleteCookies("JSESSIONID")
        )
        .rememberMe(remember -> remember
                .key(rememberMeKey)
                .tokenValiditySeconds(7 * 24 * 60 * 60)
        )
```

ClinicStatusFilter.java:

```
package com.clinicflow.security;

import com.clinicflow.domain.entity.Clinic;
import com.clinicflow.domain.enums.ClinicStatus;
import com.clinicflow.domain.enums.Role;
import com.clinicflow.domain.repository.ClinicRepository;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
```

```
import org.springframework.security.web.authentication.logout.SecurityContextLogoutH
    andler;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.Optional;

/**
 * A session created before a clinic is suspended must stop working immediately.
 * The principal only stores the clinic id, so this filter rechecks the current
 * clinic state for every clinic or doctor request.
 */
@Component
public class ClinicStatusFilter extends OncePerRequestFilter {

    private final ClinicRepository clinicRepository;
    private final SecurityContextLogoutHandler logoutHandler = new
        SecurityContextLogoutHandler();

    public ClinicStatusFilter(ClinicRepository clinicRepository) {
        this.clinicRepository = clinicRepository;
    }

    @Override
    protected boolean shouldNotFilter(HttpServletRequest request) {
        String path = request.getRequestURI();
        return !(path.startsWith("/clinic/") || path.startsWith("/doctor/")
                || path.equals("/post-login"));
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse
        response,
                                    FilterChain filterChain) throws
            ServletException, IOException {
        Authentication authentication =
```

```
            SecurityContextHolder.getContext().getAuthentication();
        if (authentication != null && authentication.getPrincipal() instanceof
            AppUserPrincipal principal
                && principal.getClinicId() != null
                && (principal.getRole() == Role.CLINIC_ADMIN || principal.getRole()
                == Role.DOCTOR)) {
            Optional<Clinic> clinic =
                clinicRepository.findById(principal.getClinicId());
            if (clinic.isEmpty() || clinic.get().getStatus() != ClinicStatus.ACTIVE)
                {
                logoutHandler.logout(request, response, authentication);
                response.sendRedirect(request.getContextPath() +
                "/login?clinicSuspended");
                return;
            }
        }
        filterChain.doFilter(request, response);
    }
}
```

PublicIdGenerator.java:

```
package com.clinicflow.common;

import java.security.SecureRandom;
import java.time.Year;
import java.util.Locale;

public final class PublicIdGenerator {

    private static final SecureRandom RANDOM = new SecureRandom();
    private static final char[] ALPHABET =
        "ABCDEFGHJKLMNPQRSTUVWXYZ23456789".toCharArray();

    private PublicIdGenerator() {
    }
```

```
    public static String patientId(long sequence) {
        return String.format(Locale.ROOT, "PAT-%d-%04d", Year.now().getValue(),
            sequence);
    }

    public static String appointmentId(long sequence) {
        return String.format(Locale.ROOT, "APT-%d-%05d", Year.now().getValue(),
            sequence);
    }

    public static String prescriptionCode() {
        StringBuilder sb = new StringBuilder(32);
        for (int i = 0; i < 32; i++) {
            sb.append(ALPHABET[RANDOM.nextInt(ALPHABET.length)]);
        }
        return sb.toString();
    }
}
```

SecretCipher.java:

```
package com.clinicflow.common;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import javax.crypto.Cipher;
import javax.crypto.spec.GCMParameterSpec;
import javax.crypto.spec.SecretKeySpec;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.SecureRandom;
import java.util.Base64;

/**
 * Small AES-GCM helper for at-rest secrets such as SMTP passwords. The key is
 * derived from the deployment secret; changing that secret makes old ciphertext
```

```
 * unreadable, so operators must re-enter SMTP credentials after rotation.
 */
@Component
public class SecretCipher {

    private static final int IV_BYTES = 12;
    private static final int TAG_BITS = 128;

    private final SecretKeySpec key;
    private final SecureRandom random = new SecureRandom();

    public SecretCipher(@Value("${clinicflow.security.secret-key:}") String
        configured,
                @Value("${clinicflow.security.remember-me-key:local-only-key
            }") String fallback) {
        String material = configured == null || configured.isBlank() ? fallback :
            configured;
        try {
            byte[] digest = MessageDigest.getInstance("SHA-256")
                    .digest(material.getBytes(StandardCharsets.UTF_8));
            this.key = new SecretKeySpec(digest, "AES");
        } catch (Exception ex) {
            throw new IllegalStateException("Could not initialise the secret
                cipher", ex);
        }
    }

    public byte[] encrypt(String plaintext) {
        if (plaintext == null || plaintext.isEmpty()) {
            return null;
        }
        try {
            byte[] iv = new byte[IV_BYTES];
            random.nextBytes(iv);
            Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
            cipher.init(Cipher.ENCRYPT_MODE, key, new GCMParameterSpec(TAG_BITS,
```

```
            iv));
        byte[] encrypted =
            cipher.doFinal(plaintext.getBytes(StandardCharsets.UTF_8));
        byte[] result = new byte[iv.length + encrypted.length];
        System.arraycopy(iv, 0, result, 0, iv.length);
        System.arraycopy(encrypted, 0, result, iv.length, encrypted.length);
        return result;
    } catch (Exception ex) {
        throw new IllegalStateException("Could not encrypt the secret", ex);
    }
}

public String decrypt(byte[] value) {
    if (value == null || value.length <= IV_BYTES) {
        return null;
    }
    try {
        byte[] iv = new byte[IV_BYTES];
        System.arraycopy(value, 0, iv, 0, IV_BYTES);
        byte[] ciphertext = new byte[value.length - IV_BYTES];
        System.arraycopy(value, IV_BYTES, ciphertext, 0, ciphertext.length);
        Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
        cipher.init(Cipher.DECRYPT_MODE, key, new GCMParameterSpec(TAG_BITS,
            iv));
        return new String(cipher.doFinal(ciphertext), StandardCharsets.UTF_8);
    } catch (Exception ex) {
        return null;
    }
}
```

AuthTokenService.java:

```
package com.clinicflow.identity;

import com.clinicflow.domain.entity.AuthToken;
```

```
import com.clinicflow.domain.entity.User;
import com.clinicflow.domain.enums.AuthTokenPurpose;
import com.clinicflow.domain.repository.AuthTokenRepository;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.Duration;
import java.time.LocalDateTime;
import java.time.ZoneOffset;
import java.util.Optional;

@Service
public class AuthTokenService {

    private final AuthTokenRepository repository;
    private final Duration verifyEmailTtl;
    private final Duration resetPasswordTtl;
    private final Duration staffInviteTtl;
    private final Duration emailChangeTtl;

    public AuthTokenService(AuthTokenRepository repository,
                            @Value("${clinicflow.auth.verify-email-ttl-hours:24}")
            long verifyEmailTtlHours,
                            @Value("${clinicflow.auth.reset-password-ttl-minutes:60}
            ") long resetPasswordTtlMinutes,
                            @Value("${clinicflow.auth.staff-invite-ttl-hours:72}")
            long staffInviteTtlHours,
                            @Value("${clinicflow.auth.email-change-ttl-hours:24}")
            long emailChangeTtlHours) {
        this.repository = repository;
        this.verifyEmailTtl = Duration.ofHours(verifyEmailTtlHours);
        this.resetPasswordTtl = Duration.ofMinutes(resetPasswordTtlMinutes);
        this.staffInviteTtl = Duration.ofHours(staffInviteTtlHours);
        this.emailChangeTtl = Duration.ofHours(emailChangeTtlHours);
    }
```

```
/**
 * Issues one active token per user/purpose. Older tokens are deleted before the
 * new token is stored, so a resend invalidates the previous link.
 */
@Transactional
public IssuedToken issue(User user, AuthTokenPurpose purpose) {
    repository.deleteByUserAndPurpose(user.getUserId(), purpose);

    String rawToken = TokenHasher.newToken();
    AuthToken token = new AuthToken();
    token.setUser(user);
    token.setPurpose(purpose);
    token.setTokenHash(TokenHasher.sha256(rawToken));
    token.setExpiresAt(now().plus(ttlFor(purpose)));
    token.setCreatedAt(now());
    repository.save(token);
    return new IssuedToken(rawToken, token);
}

/**
 * Locks the row, validates it and marks it used. Callers must be transactional
 * so the mark-used and the domain change commit together.
 */
@Transactional
public Optional<AuthToken> consume(String rawToken, AuthTokenPurpose purpose) {
    if (rawToken == null || rawToken.isBlank()) {
        return Optional.empty();
    }
    LocalDateTime now = now();
    return repository.findForUpdate(TokenHasher.sha256(rawToken), purpose)
            .filter(token -> token.getUsedAt() == null)
            .filter(token -> token.getExpiresAt().isAfter(now))
            .map(token -> {
                token.setUsedAt(now);
                return token;
```

```
            });
    }

    @Transactional(readOnly = true)
    public boolean isValid(String rawToken, AuthTokenPurpose purpose) {
        if (rawToken == null || rawToken.isBlank()) {
            return false;
        }
        LocalDateTime now = now();
        return repository.findByTokenHashAndPurpose(TokenHasher.sha256(rawToken),
            purpose)
                .filter(token -> token.getUsedAt() == null)
                .filter(token -> token.getExpiresAt().isAfter(now))
                .isPresent();
    }
```

SlotService.java:

```
package com.clinicflow.web.patient;

import com.clinicflow.domain.entity.Appointment;
import com.clinicflow.domain.entity.DoctorSchedule;
import com.clinicflow.domain.enums.AppointmentStatus;
import com.clinicflow.domain.enums.ClinicStatus;
import com.clinicflow.domain.enums.DoctorStatus;
import com.clinicflow.domain.repository.AppointmentRepository;
import com.clinicflow.domain.repository.ClinicDoctorRepository;
import com.clinicflow.domain.repository.ClinicRepository;
import com.clinicflow.domain.repository.DoctorRepository;
import com.clinicflow.domain.repository.DoctorScheduleRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
```

```
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import java.util.TreeSet;

@Service
public class SlotService {

    public static final List<AppointmentStatus> ACTIVE_STATUSES = List.of(
            AppointmentStatus.BOOKED, AppointmentStatus.CONFIRMED,
                AppointmentStatus.CHECKED_IN,
            AppointmentStatus.IN_CONSULTATION);

    private static final DateTimeFormatter TIME =
        DateTimeFormatter.ofPattern("HH:mm");

    private final ClinicRepository clinicRepository;
    private final ClinicDoctorRepository clinicDoctorRepository;
    private final DoctorRepository doctorRepository;
    private final DoctorScheduleRepository scheduleRepository;
    private final AppointmentRepository appointmentRepository;

    public SlotService(ClinicRepository clinicRepository, ClinicDoctorRepository
        clinicDoctorRepository,
                DoctorRepository doctorRepository, DoctorScheduleRepository
            scheduleRepository,
                AppointmentRepository appointmentRepository) {
        this.clinicRepository = clinicRepository;
        this.clinicDoctorRepository = clinicDoctorRepository;
        this.doctorRepository = doctorRepository;
        this.scheduleRepository = scheduleRepository;
        this.appointmentRepository = appointmentRepository;
    }
```

```
@Transactional(readOnly = true)
public DaySlots slots(Long clinicId, Long doctorId, LocalDate date) {
    if (clinicId == null || doctorId == null || date == null) {
        return new DaySlots(date, doctorId, clinicId, List.of(), 0);
    }
    var clinic = clinicRepository.findById(clinicId)
            .filter(candidate -> candidate.getStatus() == ClinicStatus.ACTIVE)
            .orElseThrow(() -> new IllegalArgumentException("Clinic is not
            available."));
    var doctor = doctorRepository.findById(doctorId)
            .filter(candidate -> candidate.getStatus() == DoctorStatus.ACTIVE)
            .orElseThrow(() -> new IllegalArgumentException("Doctor is not
            available."));
    var link =
        clinicDoctorRepository.findByClinicClinicIdAndDoctorDoctorId(clinicId,
        doctorId)
            .filter(candidate -> candidate.getStatus() == DoctorStatus.ACTIVE)
            .orElseThrow(() -> new IllegalArgumentException("Doctor is not
            available at this clinic."));

    int weekday = date.getDayOfWeek().getValue();
    List<DoctorSchedule> schedules = scheduleRepository
            .findByDoctorDoctorIdAndClinicClinicIdAndWeekday(doctorId, clinicId,
            weekday).stream()
            .filter(schedule -> schedule.getEffectiveFrom() == null
                    || !date.isBefore(schedule.getEffectiveFrom()))
            .filter(schedule -> schedule.getEffectiveTo() == null
                    || !date.isAfter(schedule.getEffectiveTo()))
            .toList();

    List<Appointment> booked = appointmentRepository
            .findByDoctorDoctorIdAndSlotStartBetweenOrderBySlotStartAsc(doctorId
            ,
                    date.atStartOfDay(), date.plusDays(1).atStartOfDay());
    Set<LocalDateTime> taken = new HashSet<>();
```

```
        for (Appointment appointment : booked) {
            if (ACTIVE_STATUSES.contains(appointment.getStatus())) {
                taken.add(appointment.getSlotStart());
            }
        }
```

BookingService.java:

```
package com.clinicflow.web.patient;

import com.clinicflow.audit.AuditService;
import com.clinicflow.common.PublicIdGenerator;
import com.clinicflow.domain.entity.Appointment;
import com.clinicflow.domain.entity.Clinic;
import com.clinicflow.domain.entity.ClinicPatient;
import com.clinicflow.domain.entity.Consultation;
import com.clinicflow.domain.entity.Doctor;
import com.clinicflow.domain.entity.Patient;
import com.clinicflow.domain.entity.Prescription;
import com.clinicflow.domain.enums.AppointmentStatus;
import com.clinicflow.domain.enums.AuditOutcome;
import com.clinicflow.domain.enums.ClinicStatus;
import com.clinicflow.domain.enums.ConsultationStatus;
import com.clinicflow.domain.enums.DoctorStatus;
import com.clinicflow.domain.enums.Role;
import com.clinicflow.domain.repository.AppointmentRepository;
import com.clinicflow.domain.repository.ClinicDoctorRepository;
import com.clinicflow.domain.repository.ClinicPatientRepository;
import com.clinicflow.domain.repository.ClinicRepository;
import com.clinicflow.domain.repository.ConsultationRepository;
import com.clinicflow.domain.repository.DoctorRepository;
import com.clinicflow.domain.repository.PatientRepository;
import com.clinicflow.domain.repository.PrescriptionRepository;
import com.clinicflow.notifications.AppointmentBookedEvent;
import com.clinicflow.notifications.AppointmentCancelledEvent;
```

```
import org.springframework.context.ApplicationEventPublisher;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;
import java.time.ZoneOffset;
import java.time.format.DateTimeFormatter;
import java.util.List;
import java.util.UUID;

@Service
public class BookingService {

    private static final DateTimeFormatter DATE_TIME =
        DateTimeFormatter.ofPattern("dd MMM yyyy, HH:mm");

    private final AppointmentRepository appointmentRepository;
    private final ClinicRepository clinicRepository;
    private final DoctorRepository doctorRepository;
    private final ClinicDoctorRepository clinicDoctorRepository;
    private final ClinicPatientRepository clinicPatientRepository;
    private final PatientRepository patientRepository;
    private final ConsultationRepository consultationRepository;
    private final PrescriptionRepository prescriptionRepository;
    private final SlotService slotService;
    private final ApplicationEventPublisher events;
    private final AuditService auditService;

    public BookingService(AppointmentRepository appointmentRepository,
        ClinicRepository clinicRepository,
                DoctorRepository doctorRepository, ClinicDoctorRepository
            clinicDoctorRepository,
                ClinicPatientRepository clinicPatientRepository,
            PatientRepository patientRepository,
                ConsultationRepository consultationRepository,
                PrescriptionRepository prescriptionRepository, SlotService
```

```
            slotService,
                    ApplicationEventPublisher events, AuditService
            auditService) {
        this.appointmentRepository = appointmentRepository;
        this.clinicRepository = clinicRepository;
        this.doctorRepository = doctorRepository;
        this.clinicDoctorRepository = clinicDoctorRepository;
        this.clinicPatientRepository = clinicPatientRepository;
        this.patientRepository = patientRepository;
        this.consultationRepository = consultationRepository;
        this.prescriptionRepository = prescriptionRepository;
        this.slotService = slotService;
        this.events = events;
        this.auditService = auditService;
    }

    @Transactional
    public Long book(Long patientUserId, BookingForm form) {
        Patient patient = patientRepository.findByUserUserId(patientUserId)
                .orElseThrow(() -> new IllegalStateException("Your patient profile
                is not linked to this account."));
        Clinic clinic = clinicRepository.findById(form.getClinicId())
                .filter(candidate -> candidate.getStatus() == ClinicStatus.ACTIVE)
                .orElseThrow(() -> new IllegalArgumentException("Clinic is not
                available."));
        // Lock the doctor row. This serialises competing bookings for the same
           doctor
        // and makes the availability check + insert atomic. The filtered unique
           index
        // on Appointments is the final database-level guard.
        Doctor doctor = doctorRepository.findForUpdate(form.getDoctorId())
                .filter(candidate -> candidate.getStatus() == DoctorStatus.ACTIVE)
                .orElseThrow(() -> new IllegalArgumentException("Doctor is not
                available."));
```

```
clinicDoctorRepository.findByClinicClinicIdAndDoctorDoctorId(clinic.getClini
    cId(),
            doctor.getDoctorId())
        .filter(link -> link.getStatus() == DoctorStatus.ACTIVE)
```

ClinicLinkRequestService.java:

```
package com.clinicflow.web.clinic;

import com.clinicflow.audit.AuditService;
import com.clinicflow.domain.entity.Appointment;
import com.clinicflow.domain.entity.ClinicPatient;
import com.clinicflow.domain.entity.Patient;
import com.clinicflow.domain.entity.PatientLinkRequest;
import com.clinicflow.domain.entity.Prescription;
import com.clinicflow.domain.entity.User;
import com.clinicflow.domain.enums.AuditOutcome;
import com.clinicflow.domain.enums.Gender;
import com.clinicflow.domain.enums.LinkRequestMethod;
import com.clinicflow.domain.enums.LinkRequestStatus;
import com.clinicflow.domain.enums.Role;
import com.clinicflow.domain.repository.AppointmentRepository;
import com.clinicflow.domain.repository.ClinicPatientRepository;
import com.clinicflow.domain.repository.PatientLinkRequestRepository;
import com.clinicflow.domain.repository.PatientRepository;
import com.clinicflow.domain.repository.PrescriptionRepository;
import com.clinicflow.domain.repository.UserRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;
import java.time.ZoneOffset;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Comparator;
```

```
import java.util.List;
import java.util.Optional;

/**
 * Clinic-side half of account-to-clinic linking. Every read is scoped by the clinic
 * id from the authenticated clinic administrator's session, never from a request
 * parameter.
 *
 * <p>Approval has two modes. Without a clinic patient ID the account's own patient
 * row becomes the clinic patient. With a clinic patient ID the account is moved
    onto
 * that existing clinic record: appointments, prescriptions and prior link requests
 * are reassigned, missing profile fields are copied across, and the account's old
 * row is retained without a login so the audit trail is never broken.</p>
 */
@Service
public class ClinicLinkRequestService {

    private static final DateTimeFormatter DATE = DateTimeFormatter.ofPattern("dd
        MMM yyyy");
    private static final DateTimeFormatter DATE_TIME =
        DateTimeFormatter.ofPattern("dd MMM yyyy, HH:mm");

    private final PatientLinkRequestRepository linkRequestRepository;
    private final ClinicPatientRepository clinicPatientRepository;
    private final PatientRepository patientRepository;
    private final AppointmentRepository appointmentRepository;
    private final PrescriptionRepository prescriptionRepository;
    private final UserRepository userRepository;
    private final AuditService auditService;

    public ClinicLinkRequestService(PatientLinkRequestRepository
        linkRequestRepository,
                                    ClinicPatientRepository clinicPatientRepository,
                                    PatientRepository patientRepository,
                                    AppointmentRepository appointmentRepository,
```

```
                        PrescriptionRepository prescriptionRepository,
                        UserRepository userRepository,
                        AuditService auditService) {
    this.linkRequestRepository = linkRequestRepository;
    this.clinicPatientRepository = clinicPatientRepository;
    this.patientRepository = patientRepository;
    this.appointmentRepository = appointmentRepository;
    this.prescriptionRepository = prescriptionRepository;
    this.userRepository = userRepository;
    this.auditService = auditService;
}

@Transactional(readOnly = true)
public ClinicLinkRequestListView list(Long clinicId) {
    List<PatientLinkRequest> requests = new ArrayList<>(
            linkRequestRepository.findByClinicClinicIdOrderByCreatedAtDesc(clini
            cId));
    requests.sort(Comparator
            .comparing((PatientLinkRequest request) -> request.getStatus() !=
            LinkRequestStatus.PENDING)
            .thenComparing(PatientLinkRequest::getCreatedAt,
            Comparator.reverseOrder()));
    long pending = requests.stream()
            .filter(request -> request.getStatus() == LinkRequestStatus.PENDING)
            .count();
    List<ClinicLinkRequestListView.Row> rows = requests.stream()
            .map(this::toRow)
            .toList();
    return new ClinicLinkRequestListView(rows, pending, requests.size());
}

@Transactional(readOnly = true)
public Optional<ClinicLinkRequestDetailView> detail(Long clinicId, Long
    requestId) {
    return linkRequestRepository.findById(requestId)
```

```
            .filter(request ->
            request.getClinic().getClinicId().equals(clinicId))
            .map(this::toDetail);
    }
```

DoctorConsultationService.java:

```
package com.clinicflow.web.doctor;

import com.clinicflow.audit.AuditService;
import com.clinicflow.domain.entity.Appointment;
import com.clinicflow.domain.entity.Consultation;
import com.clinicflow.domain.entity.ConsultationAddendum;
import com.clinicflow.domain.entity.DoctorUserLink;
import com.clinicflow.domain.entity.Prescription;
import com.clinicflow.domain.enums.AppointmentStatus;
import com.clinicflow.domain.enums.AuditOutcome;
import com.clinicflow.domain.enums.ConsultationStatus;
import com.clinicflow.domain.enums.Role;
import com.clinicflow.domain.repository.AppointmentRepository;
import com.clinicflow.domain.repository.ConsultationAddendumRepository;
import com.clinicflow.domain.repository.ConsultationRepository;
import com.clinicflow.domain.repository.DoctorUserLinkRepository;
import com.clinicflow.domain.repository.PrescriptionRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.ZoneOffset;
import java.time.format.DateTimeFormatter;
import java.util.List;

@Service
public class DoctorConsultationService {
```

```
private static final DateTimeFormatter DATE_TIME =
    DateTimeFormatter.ofPattern("dd MMM yyyy, HH:mm");

private final DoctorUserLinkRepository doctorUserLinkRepository;
private final AppointmentRepository appointmentRepository;
private final ConsultationRepository consultationRepository;
private final ConsultationAddendumRepository addendumRepository;
private final PrescriptionRepository prescriptionRepository;
private final AuditService auditService;

public DoctorConsultationService(DoctorUserLinkRepository
    doctorUserLinkRepository,
                                     AppointmentRepository appointmentRepository,
                                     ConsultationRepository consultationRepository,
                                     ConsultationAddendumRepository
            addendumRepository,
                                     PrescriptionRepository prescriptionRepository,
                                     AuditService auditService) {
    this.doctorUserLinkRepository = doctorUserLinkRepository;
    this.appointmentRepository = appointmentRepository;
    this.consultationRepository = consultationRepository;
    this.addendumRepository = addendumRepository;
    this.prescriptionRepository = prescriptionRepository;
    this.auditService = auditService;
}

@Transactional(readOnly = true)
public List<DoctorAppointmentRow> list(Long userId, LocalDate date,
    AppointmentStatus status) {
    Long doctorId = requireDoctorId(userId);
    List<Appointment> appointments;
    if (date == null) {
        appointments =
            appointmentRepository.findByDoctorDoctorIdOrderBySlotStartDesc(docto
            rId);
    } else {
```

```
        appointments =
            appointmentRepository.findByDoctorDoctorIdAndSlotStartBetweenOrderBy
            SlotStartAsc(
                doctorId, date.atStartOfDay(), date.plusDays(1).atStartOfDay());
    }
    return appointments.stream()
            .filter(appointment -> status == null || appointment.getStatus() ==
            status)
            .map(this::toRow)
            .toList();
}

@Transactional
public DoctorAppointmentDetail detail(Long userId, Long appointmentId, Long
    actorUserId) {
    DoctorUserLink link = requireDoctorLink(userId);
    Appointment appointment = requireOwned(link, appointmentId);
    Consultation consultation =
        consultationRepository.findByAppointmentAppointmentId(appointmentId)
            .orElse(null);
    Prescription prescription = consultation == null ? null
            : prescriptionRepository.findByConsultationConsultationId(consultati
            on.getConsultationId())
            .orElse(null);
    List<ConsultationAddendum> addenda = consultation == null ? List.of()
            : addendumRepository.findByConsultationConsultationIdOrderByCreatedA
            tAsc(
            consultation.getConsultationId());
    auditService.record(actorUserId, Role.DOCTOR, "CONSULTATION_VIEWED",
        "Appointment",
            appointment.getPublicAppointmentId(),
            appointment.getPatient().getPatientId(),
            AuditOutcome.SUCCESS);
    return new DoctorAppointmentDetail(appointment.getAppointmentId(),
```

appointment.getPublicAppointmentId(),
appointment.getPatient().getPatientId(),
appointment.getPatient().getPublicPatientId(),
appointment.getPatient().getDisplayName(),
appointment.getPatient().getDob(),
appointment.getPatient().getGender(),
appointment.getPatient().getMobile(),
appointment.getPatient().getEmail(),
appointment.getPatient().getBloodGroup(),
appointment.getPatient().getAllergies(),
appointment.getPatient().getConditions(),
appointment.getClinic().getName(),

PrescriptionService.java:

```
package com.clinicflow.web.prescription;

import com.clinicflow.audit.AuditService;
import com.clinicflow.common.PublicIdGenerator;
import com.clinicflow.domain.entity.Consultation;
import com.clinicflow.domain.entity.DoctorUserLink;
import com.clinicflow.domain.entity.Patient;
import com.clinicflow.domain.entity.Prescription;
import com.clinicflow.domain.entity.PrescriptionDispense;
import com.clinicflow.domain.entity.PrescriptionDispenseItem;
import com.clinicflow.domain.entity.PrescriptionItem;
import com.clinicflow.domain.enums.AuditOutcome;
import com.clinicflow.domain.enums.PrescriptionStatus;
import com.clinicflow.domain.enums.Role;
import com.clinicflow.domain.repository.ConsultationRepository;
import com.clinicflow.domain.repository.DoctorUserLinkRepository;
import com.clinicflow.domain.repository.PatientRepository;
import com.clinicflow.domain.repository.PrescriptionDispenseItemRepository;
import com.clinicflow.domain.repository.PrescriptionDispenseRepository;
import com.clinicflow.domain.repository.PrescriptionItemRepository;
```

```
import com.clinicflow.domain.repository.PrescriptionRepository;
import com.clinicflow.notifications.PrescriptionIssuedEvent;
import com.clinicflow.web.doctor.PrescriptionForm;
import org.springframework.context.ApplicationEventPublisher;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;
import java.time.ZoneOffset;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;
import java.util.UUID;

@Service
public class PrescriptionService {

    private static final DateTimeFormatter DATE = DateTimeFormatter.ofPattern("dd
        MMM yyyy");
    private static final DateTimeFormatter DATE_TIME =
        DateTimeFormatter.ofPattern("dd MMM yyyy, HH:mm");

    private final PrescriptionRepository prescriptionRepository;
    private final PrescriptionItemRepository itemRepository;
    private final PrescriptionDispenseRepository dispenseRepository;
    private final PrescriptionDispenseItemRepository dispenseItemRepository;
    private final ConsultationRepository consultationRepository;
    private final DoctorUserLinkRepository doctorUserLinkRepository;
    private final PatientRepository patientRepository;
    private final ApplicationEventPublisher events;
    private final AuditService auditService;

    public PrescriptionService(PrescriptionRepository prescriptionRepository,
                               PrescriptionItemRepository itemRepository,
                               PrescriptionDispenseRepository dispenseRepository,
                               PrescriptionDispenseItemRepository
            dispenseItemRepository,
```

```
                    ConsultationRepository consultationRepository,
                    DoctorUserLinkRepository doctorUserLinkRepository,
                    PatientRepository patientRepository,
                    ApplicationEventPublisher events,
                    AuditService auditService) {
    this.prescriptionRepository = prescriptionRepository;
    this.itemRepository = itemRepository;
    this.dispenseRepository = dispenseRepository;
    this.dispenseItemRepository = dispenseItemRepository;
    this.consultationRepository = consultationRepository;
    this.doctorUserLinkRepository = doctorUserLinkRepository;
    this.patientRepository = patientRepository;
    this.events = events;
    this.auditService = auditService;
}

@Transactional
public Long create(Long userId, Long consultationId, PrescriptionForm form, Long
    actorUserId) {
    DoctorUserLink link = doctorUserLinkRepository.findByUserUserId(userId)
            .orElseThrow(() -> new IllegalStateException("No doctor profile is
            linked to this account."));
    Consultation consultation =
        consultationRepository.findForUpdate(consultationId)
            .orElseThrow(() -> new IllegalArgumentException("Consultation not
            found."));
    if (!consultation.getAppointment().getDoctor().getDoctorId()
            .equals(link.getDoctor().getDoctorId())) {
        throw new IllegalArgumentException("Consultation not found.");
    }
    if (prescriptionRepository.existsByConsultationConsultationId(consultationId
        )) {
        throw new IllegalArgumentException("A prescription has already been
            issued for this consultation.");
```

```
        }
        List<ValidatedItem> items = validateItems(form);
        if (items.isEmpty()) {
            throw new IllegalArgumentException("Add at least one medicine with a
                quantity.");
        }
        LocalDateTime now = LocalDateTime.now(ZoneOffset.UTC);
        Prescription prescription = new Prescription();
        prescription.setConsultation(consultation);
        prescription.setPatient(consultation.getAppointment().getPatient());
        prescription.setClinic(consultation.getAppointment().getClinic());
        prescription.setDoctor(consultation.getAppointment().getDoctor());
        prescription.setIssueDate(now);
        prescription.setExpiryDate(now.plusDays(form.getValidDays()));
```

PharmacyService.java:

```
package com.clinicflow.web.pharmacy;

import com.clinicflow.audit.AuditService;
import com.clinicflow.domain.entity.Pharmacy;
import com.clinicflow.domain.entity.PharmacyMedicinePrice;
import com.clinicflow.domain.entity.PharmacyUser;
import com.clinicflow.domain.entity.PharmacyVerification;
import com.clinicflow.domain.entity.Prescription;
import com.clinicflow.domain.entity.PrescriptionDispense;
import com.clinicflow.domain.entity.PrescriptionDispenseItem;
import com.clinicflow.domain.entity.PrescriptionItem;
import com.clinicflow.domain.entity.User;
import com.clinicflow.domain.enums.AuditOutcome;
import com.clinicflow.domain.enums.DispenseStatus;
import com.clinicflow.domain.enums.PharmacyStatus;
import com.clinicflow.domain.enums.PharmacyUserRole;
import com.clinicflow.domain.enums.PharmacyUserStatus;
import com.clinicflow.domain.enums.PrescriptionStatus;
```

```
import com.clinicflow.domain.enums.Role;
import com.clinicflow.domain.enums.UserStatus;
import com.clinicflow.domain.enums.VerificationOutcome;
import com.clinicflow.domain.repository.PharmacyUserRepository;
import com.clinicflow.domain.repository.PharmacyMedicinePriceRepository;
import com.clinicflow.domain.repository.PharmacyVerificationRepository;
import com.clinicflow.domain.repository.PrescriptionDispenseItemRepository;
import com.clinicflow.domain.repository.PrescriptionDispenseRepository;
import com.clinicflow.domain.repository.PrescriptionItemRepository;
import com.clinicflow.domain.repository.PrescriptionRepository;
import com.clinicflow.domain.repository.UserRepository;
import com.clinicflow.identity.InvitationService;
import com.clinicflow.web.prescription.PrescriptionService;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.nio.charset.StandardCharsets;
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.ZoneOffset;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.HexFormat;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.UUID;

@Service
public class PharmacyService {

    private static final DateTimeFormatter DATE_TIME =
```

```
    DateTimeFormatter.ofPattern("dd MMM yyyy, HH:mm");
private static final DateTimeFormatter DAY = DateTimeFormatter.ofPattern("dd
    MMM");

private final PharmacyUserRepository pharmacyUserRepository;
private final PharmacyVerificationRepository verificationRepository;
private final PrescriptionRepository prescriptionRepository;
private final PrescriptionItemRepository itemRepository;
private final PrescriptionDispenseRepository dispenseRepository;
private final PrescriptionDispenseItemRepository dispenseItemRepository;
private final PharmacyMedicinePriceRepository priceRepository;
private final PharmacyFinanceService financeService;
private final UserRepository userRepository;
private final PrescriptionService prescriptionService;
private final InvitationService invitationService;
private final AuditService auditService;

public PharmacyService(PharmacyUserRepository pharmacyUserRepository,
                       PharmacyVerificationRepository verificationRepository,
                       PrescriptionRepository prescriptionRepository,
                       PrescriptionItemRepository itemRepository,
                       PrescriptionDispenseRepository dispenseRepository,
                       PrescriptionDispenseItemRepository
            dispenseItemRepository,
                       PharmacyMedicinePriceRepository priceRepository,
                       PharmacyFinanceService financeService,
                       UserRepository userRepository,
                       PrescriptionService prescriptionService,
                       InvitationService invitationService,
                       AuditService auditService) {
    this.pharmacyUserRepository = pharmacyUserRepository;
    this.verificationRepository = verificationRepository;
    this.prescriptionRepository = prescriptionRepository;
    this.itemRepository = itemRepository;
    this.dispenseRepository = dispenseRepository;
```

```
        this.dispenseItemRepository = dispenseItemRepository;
        this.priceRepository = priceRepository;
        this.financeService = financeService;
        this.userRepository = userRepository;
        this.prescriptionService = prescriptionService;
        this.invitationService = invitationService;
        this.auditService = auditService;
    }

    @Transactional
    public VerificationResult verify(Long userId, String rawCode, String ipAddress,
        Long actorUserId) {
        PharmacyUser pharmacyUser = requireActiveUser(userId);
        String code = rawCode == null ? "" : rawCode.trim();
        Prescription prescription =
            prescriptionRepository.findByPrescriptionCode(code)
                .filter(found -> found.getPrescriptionCode().equals(code))
                .orElse(null);
        VerificationOutcome outcome;
        String message;
        boolean canDispense = false;
```

PharmacyFinanceService.java:

```
package com.clinicflow.web.pharmacy;

import com.clinicflow.domain.entity.Pharmacy;
import com.clinicflow.domain.entity.PharmacyFinanceEntry;
import com.clinicflow.domain.repository.PharmacyFinanceEntryRepository;
import com.clinicflow.domain.repository.PharmacyUserRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.LocalDateTime;
```

```
import java.time.ZoneOffset;
import java.util.List;

@Service
public class PharmacyFinanceService {

    private final PharmacyUserRepository pharmacyUserRepository;
    private final PharmacyFinanceEntryRepository entryRepository;

    public PharmacyFinanceService(PharmacyUserRepository pharmacyUserRepository,
                                  PharmacyFinanceEntryRepository entryRepository) {
        this.pharmacyUserRepository = pharmacyUserRepository;
        this.entryRepository = entryRepository;
    }

    @Transactional(readOnly = true)
    public List<PharmacyFinanceEntry> list(Long userId) {
        return pharmacyUserRepository.findByUserUserId(userId)
                .map(link ->
                entryRepository.findByPharmacyPharmacyIdOrderByEntryDateDescCreatedA
                tDesc(
                        link.getPharmacy().getPharmacyId()))
                .orElse(List.of());
    }

    @Transactional(readOnly = true)
    public Pharmacy pharmacy(Long userId) {
        return pharmacyUserRepository.findByUserUserId(userId)
                .map(com.clinicflow.domain.entity.PharmacyUser::getPharmacy)
                .orElse(null);
    }

    @Transactional
    public void addEntry(Long userId, String type, BigDecimal amount, String
        description,
                         LocalDate entryDate, Long actorUserId) {
        addEntry(userId, type, amount, description, entryDate, actorUserId, null);
```

```
}

@Transactional
public void addEntry(Long userId, String type, BigDecimal amount, String
    description,
                    LocalDate entryDate, Long actorUserId, Long dispenseId) {
    com.clinicflow.domain.entity.PharmacyUser link =
        pharmacyUserRepository.findByUserUserId(userId)
            .orElseThrow(() -> new IllegalStateException("Pharmacy account is
            not linked."));
    if (!"INCOME".equals(type) && !"EXPENSE".equals(type)) {
        throw new IllegalArgumentException("Choose INCOME or EXPENSE.");
    }
    if (amount == null || amount.compareTo(BigDecimal.ZERO) <= 0) {
        throw new IllegalArgumentException("Enter an amount greater than
            zero.");
    }
    if (entryDate == null) {
        entryDate = LocalDate.now();
    }
    PharmacyFinanceEntry entry = new PharmacyFinanceEntry();
    entry.setPharmacy(link.getPharmacy());
    entry.setEntryType(type);
    entry.setAmount(amount);
    entry.setDescription(description == null || description.isBlank() ? null :
        description.trim());
    entry.setEntryDate(entryDate);
    entry.setCreatedBy(actorUserId);
    entry.setDispenseId(dispenseId);
    entry.setCreatedAt(LocalDateTime.now(ZoneOffset.UTC));
    entryRepository.save(entry);
}

@Transactional
public void addDispenseIncome(Long userId, BigDecimal amount, String
```

```
        description,
                        LocalDate entryDate, Long actorUserId, Long
                dispenseId) {
        addEntry(userId, "INCOME", amount, description, entryDate, actorUserId,
            dispenseId);
    }
```

EmailDeliveryService.java:

```
package com.clinicflow.notifications;

import com.clinicflow.domain.entity.EmailLog;
import com.clinicflow.domain.enums.EmailStatus;
import com.clinicflow.domain.repository.EmailLogRepository;
import com.clinicflow.web.admin.SmtpSettingsService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.data.domain.PageRequest;
import org.springframework.mail.SimpleMailMessage;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.Duration;
import java.time.LocalDateTime;
import java.time.ZoneOffset;
import java.util.List;

/**
 * Transactional outbox delivery. Kept separate from the scheduled trigger so the
 * row locks and status updates always run inside a Spring transaction proxy.
 */
@Service
public class EmailDeliveryService {
```

```
private static final Logger log =
    LoggerFactory.getLogger(EmailDeliveryService.class);

private final EmailLogRepository repository;
private final JavaMailSender mailSender;
private final SmtpSettingsService smtpSettingsService;
private final String mode;
private final String fromAddress;
private final int maxAttempts;
private final int batchSize;

public EmailDeliveryService(EmailLogRepository repository,
                            JavaMailSender mailSender,
                            SmtpSettingsService smtpSettingsService,
                            @Value("${clinicflow.mail.mode:log}") String mode,
                            @Value("${clinicflow.mail.from:no-reply@clinicflow.l
        ocal}") String fromAddress,
                            @Value("${clinicflow.mail.max-attempts:3}") int
        maxAttempts,
                            @Value("${clinicflow.mail.worker-batch:20}") int
        batchSize) {
    this.repository = repository;
    this.mailSender = mailSender;
    this.smtpSettingsService = smtpSettingsService;
    this.mode = mode;
    this.fromAddress = fromAddress;
    this.maxAttempts = maxAttempts;
    this.batchSize = batchSize;
}

@Transactional
public void dispatchBatch() {
    LocalDateTime now = LocalDateTime.now(ZoneOffset.UTC);
    List<EmailLog> due = repository.findDueForUpdate(
            EmailStatus.QUEUED, now, PageRequest.of(0, Math.max(1, batchSize)));
    if (due.isEmpty()) {
```

```
            return;
        }

        for (EmailLog email : due) {
            if (email.getAttempts() != null && email.getAttempts() >= maxAttempts) {
                email.setStatus(EmailStatus.FAILED);
                continue;
            }
            try {
                deliver(email);
                email.setStatus(EmailStatus.SENT);
                email.setSentAt(now);
                email.setLastError(null);
                email.setNextAttemptAt(null);
            } catch (RuntimeException ex) {
                int attempts = (email.getAttempts() == null ? 0 :
                email.getAttempts()) + 1;
                email.setAttempts(attempts);
                email.setLastError(truncate(ex.getMessage(), 1000));
                if (attempts >= maxAttempts) {
                    email.setStatus(EmailStatus.FAILED);
                } else {
```

V009__pharmacy_pos_pricing.sql:

```
-- ============================== POS pricing, sale totals and finance linkage
   ==============================

-- Per-pharmacy medicine price list. The POS pre-fills these prices and remembers
-- the price entered on a sale so repeat sales are fully automatic.
CREATE TABLE dbo.PharmacyMedicinePrices (
  price_id BIGINT IDENTITY(1,1) CONSTRAINT PK_PharmacyMedicinePrices PRIMARY
     KEY,
  pharmacy_id BIGINT NOT NULL CONSTRAINT FK_PMP_pharmacy REFERENCES
     dbo.Pharmacies(pharmacy_id),
```

```
  medicine NVARCHAR(200) NOT NULL,
  unit_price DECIMAL(12,2) NOT NULL CONSTRAINT DF_PMP_price DEFAULT 0
        CONSTRAINT CK_PMP_price CHECK (unit_price >= 0),
  updated_at DATETIME2(3) NOT NULL CONSTRAINT DF_PMP_updated DEFAULT
     SYSUTCDATETIME(),
  CONSTRAINT UQ_PMP_pharmacy_medicine UNIQUE (pharmacy_id, medicine)
);
CREATE INDEX IX_PMP_pharmacy ON dbo.PharmacyMedicinePrices(pharmacy_id);
GO

-- A dispense is the POS sale/receipt. Monetary columns are nullable for legacy
-- dispenses created before the POS module existed.
ALTER TABLE dbo.PrescriptionDispenses ADD
  subtotal DECIMAL(12,2) NOT NULL CONSTRAINT DF_PD_subtotal DEFAULT 0,
  discount DECIMAL(12,2) NOT NULL CONSTRAINT DF_PD_discount DEFAULT 0,
  tax_rate DECIMAL(5,2) NOT NULL CONSTRAINT DF_PD_tax_rate DEFAULT 0,
  tax_amount DECIMAL(12,2) NOT NULL CONSTRAINT DF_PD_tax_amount DEFAULT 0,
  total DECIMAL(12,2) NOT NULL CONSTRAINT DF_PD_total DEFAULT 0,
  payment_method NVARCHAR(20) NULL,
  amount_tendered DECIMAL(12,2) NULL,
  change_due DECIMAL(12,2) NULL,
  CONSTRAINT CK_PD_money CHECK (subtotal >= 0 AND discount >= 0 AND tax_rate >= 0
            AND tax_amount >= 0 AND total >= 0);
GO

ALTER TABLE dbo.PrescriptionDispenseItems ADD
  unit_price DECIMAL(12,2) NOT NULL CONSTRAINT DF_PDI_price DEFAULT 0,
  line_total DECIMAL(12,2) NOT NULL CONSTRAINT DF_PDI_total DEFAULT 0,
  CONSTRAINT CK_PDI_money CHECK (unit_price >= 0 AND line_total >= 0);
GO

ALTER TABLE dbo.PharmacyFinanceEntries ADD
  dispense_id BIGINT NULL CONSTRAINT FK_PFE_dispense REFERENCES
     dbo.PrescriptionDispenses(dispense_id);
GO
```

```
CREATE INDEX IX_PFE_dispense ON dbo.PharmacyFinanceEntries(dispense_id) WHERE
    dispense_id IS NOT NULL;
GO

PRINT 'IMS POS pricing and sale totals created.';
GO
```

6.4 REFERENCES:

[1] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman. Role-based access control models. IEEE Computer, 29(2):38–47, Feb. 1996.

[2] C. Walls. Spring in Action, 6th Edition. Manning Publications, Shelter Island, NY, USA, 2022.

[3] L. Spilca. Spring Security in Action, 2nd Edition. Manning Publications, Shelter Island, NY, USA, 2024.

[4] C. Bauer, G. King, and G. Gregory. Java Persistence with Spring Data and Hibernate. Manning Publications, 2023.

[5] M. Kleppmann. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O'Reilly Media, Sebastopol, CA, USA, 2017.

[6] I. Ben-Gan. T-SQL Fundamentals (Microsoft SQL Server 2022), 4th Edition. Microsoft Press, Redmond, WA, USA, 2023.

[7] S. K. Porterfield, A. Deshmukh, and L. Narayanan. Preventing prescription forgery and multi-pharmacy over-dispensing through tokenized electronic prescriptions. Journal of Biomedical Informatics, 128:104–116, Apr. 2022.

[8] S. J. Grannis, H. Xu, J. R. Vest, and K. T. Marsolo. Evaluating master patient index linkage accuracy and human-in-the-loop chart consolidation in outpatient networks. Journal of the American Medical Informatics Association (JAMIA), 27(9):1381–1390, Sep. 2020.

[9] C. Sinsky, J. D. Halamka, and R. Koppel. Medico-legal integrity of electronic clinical documentation: Draft-final lifecycles and signed addenda. International Journal of Medical Informatics, 131:103958, Nov. 2019.

[10] M. Dworkin. Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. NIST Special Publication 800-38D, National Institute of Standards and Technology, 2007.

[11] N. Provos and D. Mazieres. A future-adaptable password scheme (BCrypt). Proceedings of the USENIX Annual Technical Conference, pages 81–91, Monterey, CA, USA, 1999.

[12] C. Richardson. Microservices Patterns: With Examples in Java (Transactional Outbox Pattern). Manning Publications, 2019.

[13] J. Bloch. Effective Java, 3rd Edition. Addison-Wesley Professional, Boston, MA, USA, 2018.

[14] R. S. Pressman and B. R. Maxim. Software Engineering: A Practitioner's Approach, 9th Edition. McGraw-Hill Education, New York, NY, USA, 2020.

[15] M. Thirunavukkarasu and S. Venkatesh. Integrated retail pharmacy point-of-sale automation and real-time financial cashbook reconciliation. International Journal of Healthcare Technology and Management, 20(3):211–228, 2023.

[16] OWASP Foundation. OWASP Application Security Verification Standard (ASVS) Version 4.0.3 and Top 10 Web Application Security Risks. Open Worldwide Application Security Project, 2022.

[17] D. W. Bates, K. E. Walsh, and R. Kaushal. Standardizing outpatient clinic drug formularies to prevent prescribing errors. BMJ Quality & Safety, 30(6):472–480, 2021.

[18] National Health Authority (NHA). Ayushman Bharat Digital Mission — Electronic Prescription and Health Information Exchange Blueprint. Government of India, 2023.

[19] VMware / Broadcom. Spring Boot 4 & Spring Framework Reference Documentation. https://docs.spring.io/spring-boot/index.html, accessed Oct. 2026.

[20] Microsoft Corporation. Microsoft SQL Server 2022 Technical Documentation — Binary Collations, Filtered Unique Indexes, and ROWVERSION. https://learn.microsoft.com/en-us/sql/sql-server/, accessed Oct. 2026.

[21] Redgate Software. Flyway Database Schema Migrations Documentation. https://documentation.red-gate.com/fd/, accessed Oct. 2026.

[22] Thymeleaf Project. Using Thymeleaf 3.1 & Spring Security Integration Reference. https://www.thymeleaf.org/documentation.html, accessed Oct. 2026.